

DNS-Anchored zk-SNARK Proofs: A Stateless Alternative to ACME Challenge-Response for Domain Control Validation

Abhinav Vishnu  *Independent Researcher*
abhinav.vishnu@icloud.com

Abstract—Domain Control Validation (DCV) is the cornerstone of trust on the web, serving as the prerequisite for issuing TLS certificates and asserting identity. The current industry standard, the Automated Certificate Management Environment (ACME) protocol, relies on synchronous, interactive challenge-response mechanisms (e.g., HTTP-01) that necessitate active server infrastructure and open network ports. This architectural requirement imposes significant friction on modern serverless, static, and air-gapped deployments, often forcing the exposure of sensitive infrastructure solely for validation purposes.

This paper presents the Portable Trust eXtensible (PTX) protocol, a novel mechanism for asynchronous, non-interactive DCV. PTX decouples the assertion of control from the delivery mechanism by utilizing Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs). We introduce a circuit design that cryptographically binds a set of ephemeral secrets (a nullifier and secret key) to a scoped metadata payload containing audience restrictions and expiration parameters anchored to the public DNS via a lightweight TXT record.

This approach eliminates the need for an active web server during validation. A prover generates a self-contained, portable, and purely stateless proof artifact that can be verified client-side by any relying party, with revocation handled via $O(\text{TTL})$ DNS record deletion. We implement a reference toolchain using the Groth16 proving system and the Poseidon hash function, achieving a circuit complexity of just 1,756 constraints and sub-15ms verification times on consumer hardware. Our security analysis demonstrates that PTX effectively mitigates replay attacks through context-commitment public inputs while offering a privacy-preserving alternative to X.509 certificate chains for identity assertions in decentralized environments.

Index Terms—zero knowledge proofs, domain name system, public key infrastructure, network security, authentication protocols, serverless computing, digital signatures, cryptography

I. INTRODUCTION

The integrity of the modern web relies fundamentally on the binding between a digital identity and a network identifier. Whether issuing X.509 certificates for TLS, verifying software supply chains, or authenticating institutional metadata, the prerequisite step is invariably Domain Control Validation (DCV). For the past decade, the automatic standard for this process has been the Automated Certificate Management Environment (ACME) protocol [1], typically utilized via the HTTP-01 or DNS-01 challenge mechanisms.

While ACME successfully automated the issuance of certificates, it introduced a significant architectural dependency: synchronicity. The HTTP-01 challenge requires the prover to run an active web server, listen on port 80, and serve a

specific token in response to a verifier’s request. This interactive “challenge-response” model creates substantial friction in modern infrastructure paradigms. Serverless functions, static site generators, air-gapped signing keys, and decentralized applications often lack the persistent, publicly addressable servers required to satisfy these synchronous challenges. Furthermore, the requirement for public addressability introduces a privacy vulnerability: active probing by third parties can reveal validation artifacts, leaking metadata regarding verification attempts without the consent of the domain owner.

This paper proposes a paradigm shift from interactive challenge-response to asynchronous, non-interactive proof. We introduce the Portable Trust eXtensible (PTX) protocol, a system that utilizes zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs) to decouple the assertion of domain control from the verification infrastructure.

In the PTX model, a domain owner generates a cryptographic proof attesting that they know a private secret corresponding to a public commitment anchored in the domain’s DNS TXT records. This proof is bound to a specific context defining the audience, scope, and validity window within the arithmetic circuit itself. The resulting artifact is a portable file that can be transmitted via any medium (email, API, or static file) and verified non-interactively by any party without requiring the prover to maintain active infrastructure, instead utilizing the Domain Name System.

By anchoring the root of trust in the Domain Name System (DNS) rather than a Certificate Authority, PTX achieves $O(\text{TTL})$ revocation via simple record deletion and enables a “serverless” validation workflow. We present a reference implementation utilizing the Groth16 proving system [2] and the Poseidon hash function [3], demonstrating that it is possible to generate secure, context-bound identity assertions with a circuit complexity of only 1,756 constraints and native verification times under 15 ms and browser-based verification under 250 ms. This approach offers a robust, privacy-preserving alternative to traditional challenge-response verification for the next generation of decentralized and ephemeral internet identity infrastructure.

The hierarchical trust model of the modern internet is predicated on a single, foundational axiom: the verifiable binding between a public key and a unique network identifier, typically a Fully Qualified Domain Name (FQDN). This binding forms the bedrock upon which nearly all digital security primitives are built, from the TLS certificates that secure trillions of dollars in daily commerce [4] to the signed software

artifacts that underpin secure supply chains [5]. Before any trusted assertion can be made about a domain, the entity making the assertion must first prove its administrative control over that domain. This prerequisite process is known as Domain Control Validation (DCV).

A failure in the integrity of DCV has catastrophic consequences. A malicious actor capable of illegitimately passing a DCV check for a target domain could be issued a valid X.509 certificate for that domain, enabling transparent man-in-the-middle (MITM) attacks against all of its users. Similarly, they could sign malicious software updates that would be accepted as authentic by package managers, or forge identity tokens that would be trusted by federated authentication systems. Consequently, the security and reliability of the DCV mechanism itself is not merely a procedural step, but a critical pillar supporting the entire public key infrastructure (PKI) of the web. The robustness of this initial validation directly dictates the integrity of all subsequent cryptographic assertions.

To address the friction of manual certificate issuance, the Internet Engineering Task Force (IETF) standardized the Automated Certificate Management Environment (ACME) protocol [1], which has since become the de facto standard for automated DCV, most notably through the Let’s Encrypt service [6]. ACME operates on an interactive challenge-response model. A certificate authority (CA) acting as the verifier issues a challenge to the domain owner (the prover), who must fulfill it to demonstrate control. The most prevalent mechanism, the HTTP-01 challenge, requires the prover to provision a specific, ephemeral token at a publicly accessible HTTP endpoint on the target domain. An alternative, the DNS-01 challenge, requires creating a temporary TXT record with a specific value.

While effective in automating DCV, this model imposes a critical architectural constraint: synchronicity. The prover’s infrastructure must be online, publicly addressable, and capable of responding to the verifier’s challenge within a short, prescribed time window. This synchronous dependency fundamentally tethers the act of validation to the operational state of live server infrastructure. This creates significant friction for a growing class of modern architectures, including serverless functions, static site deployments (e.g., Vercel, Netlify), and systems utilizing air-gapped or offline signing keys, which inherently lack the persistent, interactive endpoints that ACME demands.

Beyond the architectural constraints, the public nature of ACME’s challenge artifacts creates a significant and often overlooked privacy vulnerability. The use of a standardized, well-known URI (`/well-known/acme-challenge/`) for HTTP-01 challenges establishes a publicly observable side channel. Malicious or merely curious third parties can actively probe these endpoints on high-value domains to detect ongoing validation attempts. A successful probe does not merely reveal that a validation is occurring; it leaks sensitive metadata about a domain’s operational activities. For a commercial entity, this could prematurely reveal a confidential partnership or service integration.

The implications are more severe for operators of sensitive or anonymous platforms, such as political dissidents or investigative journalists. For these users, the act of placing a

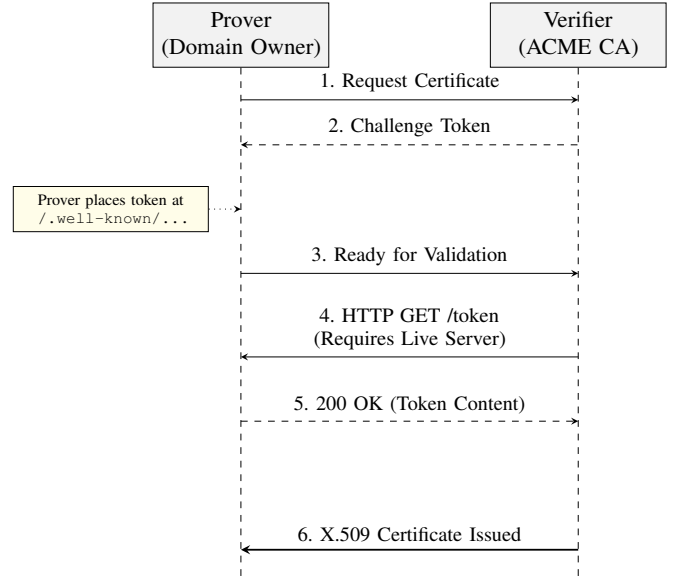


Fig. 1: The synchronous challenge-response flow of the ACME HTTP-01 protocol, requiring an interactive, publicly accessible server.

public validation token on their site constitutes an unacceptable form of forced de-anonymization, potentially exposing them to significant personal risk. The current ecosystem forces a direct and often dangerous trade-off: to achieve cryptographic verifiability, a domain owner must sacrifice a degree of operational privacy and pseudonymity.

This paper proposes a paradigm shift from interactive challenge-response to asynchronous, non-interactive proof. We introduce the **Portable Trust eXtensible (PTX)** protocol, a system that utilizes Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) to decouple the assertion of domain control from the verification infrastructure.

In the PTX model, a domain owner generates a cryptographic proof attesting that they know a private secret corresponding to a public commitment anchored in the domain’s DNS TXT records. Crucially, this proof is not a generic statement of ownership; it is bound to a specific context—defining the audience, scope, and validity window—via public inputs to the arithmetic circuit itself. The resulting artifact is a portable, self-contained file that can be transmitted via any medium (e.g., an API submission, an email attachment) and verified non-interactively by any party. This process requires no active, publicly-addressable infrastructure on the prover’s side at the time of verification, only the pre-existing DNS record. This fundamental architectural change resolves the synchronicity dependency of ACME and eliminates the attack surface for metadata leakage inherent in public challenge protocols. Furthermore, the protocol is purely stateless, requiring no long-term retention of private key material or persistent account state once the proof has been generated and anchored.

1) Protecting Owner Anonymity: The limitations of existing DCV models are particularly acute for operators of politicized or anonymous platforms, such as dissidents, activists, or investigative journalists. These individuals often need to establish trusted attribution for their work to a specific institution—for instance, a university as part of an application, or a journalistic

body for verification—without making a public, and potentially dangerous, declaration of their identity.

The PTX protocol is architected to address this critical use case by enabling a form of selective, asymmetric disclosure. A prover can generate a PTX proof intended for a single, specific audience (e.g., disclosure.reuters.com, *.harvard.edu), transmitting the proof artifact privately. The corresponding public anchor, which is a pseudorandom (SHA256) hash in a DNS hostname (padded with alphanumeric characters to ensure valid syntax), contains no correlatable information about the prover, the verifier, or the purpose of the proof, rendering the verification event itself invisible to outside observers. Upon successful verification, the prover can delete the DNS record, thereby severing the cryptographic link in $O(\text{TTL})$ time and leaving no persistent, discoverable public record of the event. This capability transforms PTX from a purely technical mechanism into a powerful tool that can empower and protect vulnerable speakers in high-stakes environments.

II. BACKGROUND AND PRELIMINARIES

This section provides a concise overview of the foundational technologies upon which it is built, in order to fully contextualize the design and security contributions of the PTX protocol. We begin by examining the operational model of the existing X.509 Public Key Infrastructure and the ACME protocol, detailing the specific architectural constraints that motivate our work. We then introduce the core cryptographic primitive, Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs), with a focus on the Groth16 proving system due to its properties of small proof size and rapid verification. Subsequently, we discuss the role of ZK-friendly hash functions, specifically the Poseidon hash function, which is critical for achieving an efficient arithmetic circuit. Finally, we review the properties of the Domain Name System (DNS) that allow it to function as a globally distributed, verifiable, albeit constrained, data store for anchoring public commitments.

A. X.509/PKI and ACME Ecosystem

The Web Public Key Infrastructure (PKI) is a hierarchical trust model designed to securely bind public keys to identities, which are typically represented by domain names. The foundational standard for this system is X.509, which defines the format of public key certificates [7]. At the apex of this hierarchy are a small number of trusted, root Certificate Authorities (CAs) whose public keys are pre-distributed in operating systems and browsers. To obtain a certificate, a subscriber (i.e., a domain owner) must prove their identity to a CA or one of its subordinate Registration Authorities (RAs). The critical gatekeeping step in this process is Domain Control Validation (DCV).

Historically, DCV was a manual, slow, and often costly process, creating a significant barrier to the widespread adoption of encrypted communications via TLS. The advent of the Automated Certificate Management Environment (ACME) protocol [1] radically transformed this landscape by providing an extensible framework for the automated issuance and renewal of certificates.

ACME’s primary innovation lies in its automated DCV mechanisms. The most common of these is the HTTP-01 challenge. In this flow, the ACME server provides the client with a random token. The client must then make this token available at a specific, well-known URI on its web server (e.g., `http://example.com/.well-known/acme-challenge/ $\tau$ token $\tau$`). The ACME server performs an HTTP GET request to this URI to verify the client’s control. This mechanism, while effective, explicitly requires the domain to be served by a publicly accessible web server capable of responding to requests on port 80.

The HTTP-01 challenge is not the only DCV method supported by ACME. Other methods include the DNS-01 challenge, which allows a client to prove control over a domain by adding a specific TXT record to its DNS zone, and the TLS-ALPN-01 challenge, which validates control by ensuring that a specific TLS extension is present in the server’s certificate chain, not achievable by most providers and is not feasible for a first-time certificate issuance.

The DNS-01 challenge, although alleviating the need for active web infrastructure for serving the proof itself, still necessitates programmatic access to the DNS zone of the domain in question, and more crucially, it is a synchronous and interactive process, requiring the client to actively perform the validation operation at the time of issuance, and can usually leak metadata as most services tend to configure a human-readable record.

Both mechanisms form the foundation of the current DCV ecosystem, and their inherent architectural limitations are the primary motivation for the work presented in this paper.

B. Zero Knowledge Proofs using Groth16

A Zero-Knowledge Proof (ZKP) is a cryptographic protocol that allows a prover to convince a verifier of the truth of a mathematical statement without revealing any information beyond the statement’s validity. The statement is typically expressed as a computational problem, often an arithmetic circuit, and the prover’s secret knowledge is a “witness” that provides a satisfying assignment to the circuit’s inputs (or “wires”). Any ZKP system must satisfy three core properties:

- **Soundness:** If the statement is false, the prover cannot convince the verifier of its truth with non-negligible probability.
- **Completeness:** If the statement is true, the prover can always convince the verifier of its truth with high probability.
- **Zero-Knowledge:** The prover cannot reveal any information beyond the statement’s validity.

For practical systems, zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) offer two additional, critical properties: they are **succinct**, meaning the proofs are very small and fast to verify, regardless of the complexity of the original computation; and they are **non-interactive**, meaning the proof is a single message that can be sent from the prover to the verifier without any back-and-forth communication.

Among the various zk-SNARK constructions, we selected the **Groth16** proving system [2] for our reference implementation. Groth16 is notable for generating extremely succinct proofs (consisting of just **three group elements** in the underlying elliptic curve) and for its highly efficient, constant-time verification algorithm. These properties are exceptionally well-suited for the PTX protocol, where proofs are intended to be portable, stored, and verified in resource-constrained or high-throughput environments.

A key consideration of the Groth16 scheme is its requirement for a per-circuit trusted setup. This ceremony generates a set of public parameters (the proving and verification keys) derived from a secret value, often termed “toxic waste.” If this secret is not securely destroyed after the ceremony, it can be used to forge proofs for that specific circuit. While this imposes a significant security assumption, widely-used public ceremonies such as the Perpetual Powers of Tau [8] have been established to mitigate this risk for a broad class of circuits by distributing the trust among many independent participants.

C. The Poseidon Hash Function

A core requirement of the PTX protocol is the ability to compute cryptographic commitments within the arithmetic circuit. Standard hash functions such as SHA-256, while secure and efficient on conventional CPUs, are notoriously inefficient when expressed as an arithmetic circuit. Their reliance on bitwise operations (XOR, AND, rotations) translates into a prohibitively large number of constraints in an R1CS (Rank-1 Constraint System) environment, rendering them impractical for succinct ZKPs.

To overcome this, a class of “ZK-friendly” hash functions have been developed, designed from the ground up to be efficient within arithmetic circuits. These functions are constructed using algebraic operations, such as addition, multiplication, and exponentiation over a prime field, which are “cheap” to express as constraints. For the PTX reference implementation, we utilize the Poseidon hash function [3], which has emerged as a de facto standard for ZK applications due to its strong security arguments and exceptional performance.

Poseidon operates as a sponge construction, applying a series of permutation rounds to an internal state. Each round is composed of three simple, arithmetically-friendly operations: adding a round constant, applying a non-linear S-Box (typically x^α for a small α), and multiplying by a Maximum Distance Separable (MDS) matrix to ensure diffusion. This algebraic structure allows for a highly optimized representation in R1CS. The use of Poseidon in our `ContextHasher` and `CredentialHasher` templates is a primary contributor to the low overall constraint count (1756) of the final `ProofDoH` circuit, making proof generation fast and the trusted setup parameters compact. The `circomlib` library provides a well-vetted and standardized implementation of Poseidon, which we use in our work [9].

D. DNS as a Verifiable Data Store

While primarily known for resolving human-readable domain names to IP addresses, the Domain Name System (DNS)

is architecturally a globally distributed, hierarchical, and eventually consistent key-value database [10]. The “key” in this system is a fully qualified domain name (FQDN), and the “value” is the data contained within a specific resource record type associated with that FQDN. For the purposes of storing arbitrary data, the TXT record type is particularly suitable, as it is explicitly designed to hold human-readable or machine-readable text strings.

However, DNS is a constrained environment, and these constraints directly inform the design of the PTX protocol. The two most significant limitations are:

- **Label Length:** Each label (the segment between two dots in a domain name) is restricted to a maximum of 63 characters. This makes it impossible to store a full 256-bit hash in its standard hexadecimal representation (64 characters). To overcome this, PTX encodes the hash of the commitment using a more compact, DNS-safe alphabet such as Base32, which respects this limit.
- **Record Size:** While TXT records can hold up to several kilobytes, they are not suited for storing large, arbitrary payloads like the full metadata of a proof. Therefore, PTX stores only a cryptographic commitment—the SHA-256 hash of the full metadata—in the TXT record.

These properties make DNS a viable, if constrained, platform for anchoring public attestations. Its ubiquity, distributed nature, and the security guarantees of DNSSEC provide the necessary foundation for the “A” (Anchor) in the PTX architecture.

III. THE PTX PROTOCOL

Having established the limitations of existing DCV mechanisms and the foundational technologies required, this section details the complete design and architecture of the Portable Trust eXtensible (PTX) protocol. Our system is designed around the core principles of non-interactivity, portability, and context-binding. It consists of three primary actors — the **Prover**, the **Anchor**, and the **Verifier** — and a single, self-contained cryptographic artifact: the **PTX file**. We begin by presenting a high-level overview of the protocol’s workflow, followed by a detailed specification of the PTX file format. Subsequently, we analyze the structure of the signed metadata payload, which is critical for preventing replay attacks. We then provide a wire-by-wire walkthrough of the ProofDoH arithmetic circuit, which forms the cryptographic heart of the system. Finally, we detail the “**unfindable anchor**” mechanism, which leverages DNS hostnames to provide a private and deniable method for publishing public commitments.

A. Architecture Overview

The PTX protocol operates as an asynchronous, three-party interaction facilitated by a single cryptographic artifact. The three logical actors in the system are the Prover, who generates the proof; the Anchor, which is the public DNS infrastructure; and the Verifier, who validates the proof. The workflow, depicted in Figure 2, is divided into a one-time setup phase and a subsequent, repeatable verification phase.

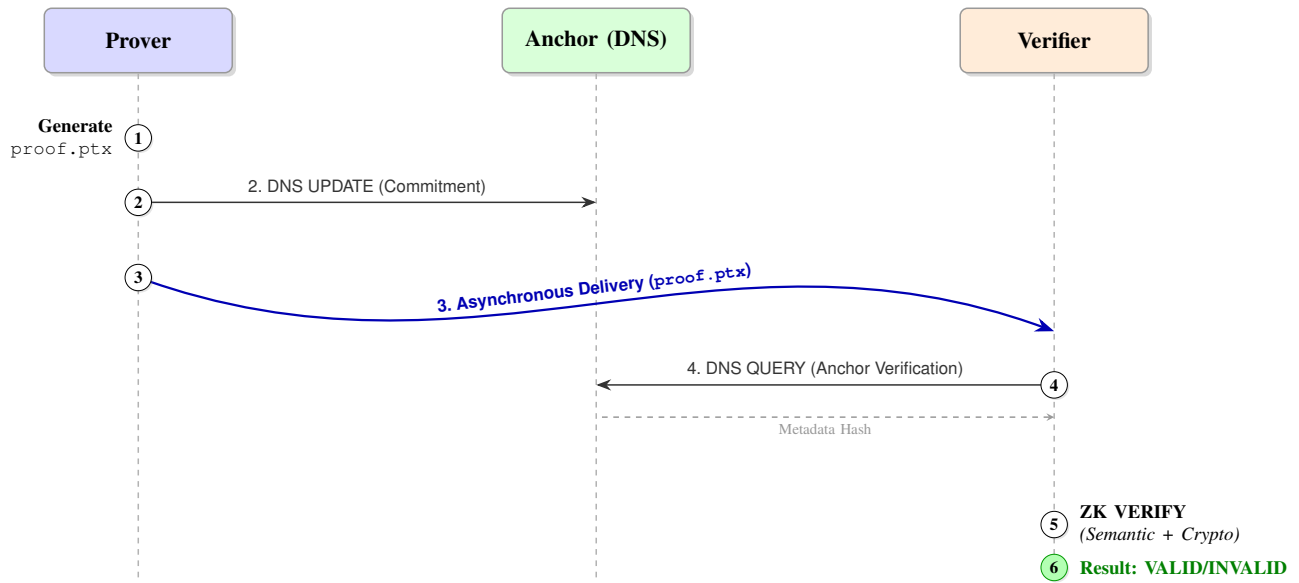


Fig. 2: The PTX protocol workflow. The use of a DNS-anchored commitment allows for non-interactive, asynchronous validation that does not require the Prover’s server to be online during the verification phase.

1) *Proof Generation and Anchoring (Prover)*: The process is initiated by the Prover. They first define the full public context of the assertion by constructing the `SignedMetadataPayload`, which includes the target domain, scope, audience, and other contextual parameters. Using their private secrets (a nullifier and a secret), the Prover generates a zk-SNARK proof that is cryptographically bound to a hash of this metadata. The Prover then computes two distinct public hashes:

- The Anchor Hostname Hash: A hash of the commitment, which is encoded into a DNS-safe string (e.g., Base32) to form a unique, pseudorandom hostname.
- The Metadata Hash: A SHA-256 hash of the full metadata payload.

The Prover performs a one-time update to the Anchor (their domain’s DNS zone), creating a TXT record at the derived Anchor Hostname containing the Metadata Hash as its value. Finally, the Prover packages the zk-SNARK proof, the full metadata, and the anchor details into the final `proof.ptx` file.

2) *Verification (Verifier)*: At any later time, the Verifier can validate the `proof.ptx` file non-interactively. The verification process consists of two parallel and independent checks:

- **Anchor Verification**: The Verifier parses the PTX file, re-derives the Anchor Hostname and the expected Metadata Hash from the enclosed metadata, and performs a DNS query. It asserts that the TXT record exists at the derived hostname and that its content exactly matches the expected hash. This confirms the proof’s “liveness” and the Prover’s control over the domain at the time the record was published.
- **Cryptographic Verification**: The Verifier extracts the zk-SNARK proof and the public signals from the PTX file. It then runs the Groth16 verification algorithm, asserting that the proof is mathematically valid for the given public inputs. This confirms the integrity of the assertion itself.

The overall validation is successful only if both the anchor verification and the cryptographic verification pass. This bimodal structure ensures that the proof is not only mathematically sound but also authoritatively anchored to the correct, live entity.

3) *Statelessness and Ephemeral Keys*: A defining characteristic of the PTX protocol is its purely stateless nature. Unlike ACME, which requires the maintenance of persistent account keypairs and session state, PTX operates without the need for long-term key retention. Each proof generation is an independent event utilizing ephemeral secrets. Once the proof is generated and the anchor is published, no further state or key material needs to be maintained by the Prover. While a Verifier may optionally maintain state to enforce one-time use via a challenge-nonce included in the context, this is not a core protocol requirement. The Prover remains strictly stateless, making this architecture particularly well-suited for ephemeral environments like serverless functions and automated CI/CD pipelines.

B. Container Design

The PTX artifact is designed to be a portable, self-contained, and extensible binary file. To ensure robust cross-platform compatibility and ease of parsing, the file format is built upon Google’s Protocol Buffers (Protobuf) [11], a language-agnostic, binary serialization format. A valid PTX file consists of two components: a fixed 5-byte magic header for immediate file type identification, followed by the serialized Protobuf message.

1) *Magic Header*: Every PTX file begins with the 5-byte sequence `50 54 58 AB 01`. This corresponds to the ASCII string “PTX”, followed by an unprintable byte (`0xAB`) and a version byte (`0x01`). This header allows file systems and parsing tools to quickly and unambiguously identify the file as a PTX v1 artifact.

2) *Protobuf Schema*: The core payload is defined by the `PtxFile` Protobuf message, detailed in Listing 1. The schema is designed for both clarity and extensibility.

Listing 1: "The PTX file format as defined in Protobuf"

```
message PtxFile {
  TrustMethod trust_method = 1;
  ZkProof proof = 2;
  string signed_metadata = 3;
  oneof anchor {
    DohAnchor doh_details =
      4;
  }
  IssuerSignature issuer_signature
    = 6;
}
```

3) Key Schema Features:

- `trust_method`: An enum that specifies the anchoring mechanism (e.g., DOH for DNS). This allows the protocol to be extended to other public data sources in the future.
- `proof`: A nested message containing the raw zk-SNARK proof data, the proving system identifier (e.g., GROTH16), and an ID for the required verification key. This provides the verifier with all necessary cryptographic context.
- `signed_metadata`: A string field containing the full, human-readable metadata payload. The hash of this field serves as the primary public input to the ZK circuit.
- `anchor`: A `oneof` field that provides a type-safe union for different anchor types. For DOH, this contains the `DohAnchor` message with the domain name.
- `issuer_signature`: An optional field for platform attestation, allowing an intermediary (e.g., an application platform) to co-sign the proof, adding a layer of institutional trust.

This Protobuf-based design provides a highly structured, efficient, and forward-compatible foundation for the PTX artifact, eliminating the parsing ambiguity common in manually-chunked binary formats.

C. Context-Bindings for Security

A raw, decontextualized zero-knowledge proof is a significant security risk. A valid proof that simply attests to knowledge of a secret is analogous to a blank, signed check: it is authentic but can be dangerously repurposed. An adversary could intercept a valid proof generated for one purpose (e.g., authenticating to a low-security forum) and replay it in a different, high-stakes context (e.g., authorizing a transaction), a classic form of replay attack.

To mitigate this fundamental threat, the PTX protocol mandates that every proof be cryptographically bound to a specific, unambiguous context. This is achieved through the `SignedMetadataPayload`, a structured JSON object that contains the full set of human-readable and machine-readable parameters for the assertion. While we recommend this payload be wrapped in a JSON Web Signature (JWS) [12] for an additional layer of traditional signature-based integrity, the

core of the ZK security model relies on hashing this payload and using the result as a public input to the circuit.

The most critical component of this payload is the `purpose` object, which is designed to be rigorously validated by the verifier. It contains the following security-critical fields:

- `scope`: A machine-readable string that defines the intended action or permission being granted (e.g., `resume:authentication`). This prevents a proof intended for one type of action from being used for another.
- `audience`: An identifier for the intended recipient(s) of the proof (e.g., `*.harvard.edu`). A verifier **MUST** reject any proof where it is not listed in the audience, which is the primary defense against cross-service replay attacks.
- `nonce`: A unique, single-use value, typically provided by the verifier for a specific session. This prevents an adversary from replaying a valid proof to the same verifier for a new session.
- `expirationTimestamp`: A Unix timestamp after which the proof must be rejected. This strictly limits the lifetime of the proof, reducing the window of opportunity for misuse if a proof artifact is compromised.

The entire `SignedMetadataPayload`, including the `purpose` object and any other data like the `entities` array, is canonicalized into a UTF-8 string and hashed using SHA-256. This 256-bit hash is then split into two 128-bit field elements (`metadataHashp1` and `metadataHashp2`) and provided as public inputs to the `ProofDoH` circuit. This design makes the mathematical validity of the zk-SNARK proof inextricably and contingently linked to the exact, bit-for-bit content of the metadata. Any tampering with the metadata payload, no matter how minor, will produce a different hash, causing the ZK proof verification to fail.

D. ProofDoH Circuit Walkthrough

The cryptographic core of the PTX protocol is the `ProofDoH` arithmetic circuit, implemented in the Circom 2.0 language [9]. The circuit's primary function is to generate a proof for the following statement: "I, the prover, know a secret `nullifier` and a secret `secret`, which, when cryptographically combined with the public context (the domain, metadata hash, etc.), correctly produce the public `commitment` and `nullifierHash`." The prover's secret inputs constitute the witness for the computation. To maintain clarity and modularity, the `ProofDoH` circuit is composed of two primary sub-circuits, or templates: the `ContextHasher` and the `CredentialHasher`.

1) *ContextHasher—Public State Binder*: The first step within the circuit is to compress all public contextual information into a single, collision-resistant cryptographic value. This ensures that the entire proof is immutably bound to one specific context. The `ContextHasher` template is responsible for this task.

It accepts four public signal inputs: `fqdn`, `metadataHash_p1`, `metadataHash_p2`, and `trustMethod`. As established in Section III.C, the 256-bit SHA-256 hash of the metadata is split into two 128-bit chunks (`_p1`, `_p2`) to be compatible with the ~254-bit

prime field of the BN254 curve [13] used by Groth16. The template instantiates a Poseidon (4) component, which hashes these four field elements into a single output field element, `contextHash`. This hash serves as a succinct, computationally efficient representation of the entire public state of the proof.

2) *CredentialHasher—Commitment Generator*: The `CredentialHasher` template implements the core commitment scheme of the protocol. It takes the prover’s private witness—the nullifier and secret signals—along with the `contextHash` from the previous step, and generates the two primary public outputs of the proof.

This template utilizes two distinct Poseidon hashers for two different purposes:

- `nullifierHasher = Poseidon (1)`: This component computes the `nullifierHash` by hashing only the private `nullifier` input. The nullifier is a secret random value chosen by the prover to ensure that each proof they generate is unique, even with the same secret. The `nullifierHash` can be made public without revealing the nullifier itself. In systems that require preventing double-spending or duplicate actions, this public hash can be stored in a record of “spent” nullifiers.
- `commitmentHasher = Poseidon (3)`: This component computes the final commitment. It takes three inputs, the `nullifier`, the `secret`, and the `contextHash`, and hashes them together. This operation cryptographically binds the prover’s private credentials to the specific public context. The resulting commitment serves as the public “lock” that the prover later proves they can open.

3) *Constraint System and Public Signal Definition*: The final `ProofDoH` template orchestrates the data flow between these components and establishes the critical constraints that form the basis of the proof. After instantiating the sub-circuits and wiring their inputs and outputs, the template defines two core equality constraints using Circom’s `===` operator.

It is critical to note that `===` is not an assignment but an assertion. It adds a constraint to the R1CS that forces the left-hand side to be equal to the right-hand side. The two key constraints are:

- `nullifierHash === credHasher.nullifierHash`
- `commitment === credHasher.commitment`

These constraints assert that the publicly supplied `nullifierHash` and `commitment` values are exactly equal to the values that were just computed inside the circuit using the prover’s private witness. If the prover does not know the correct nullifier and secret, they cannot generate a witness that satisfies these constraints, and the proof generation will fail.

Finally, the main component of the circuit declares its full list of public signals: `[nullifierHash, commitment, fqdn, metadataHash_p1, metadataHash_p2, trustMethod]`. These are the values that are embedded in the final proof and are used by the verifier’s algorithm to check its validity.

E. The Anonymous Anchor Mechanism

A core privacy goal of the PTX protocol is to prevent the public enumeration or discovery of verification events. If the DNS anchor were placed at a predictable, well-known location (e.g., `ptx-proof.example.com`), it would create a publicly discoverable side channel, mirroring the metadata leakage vulnerability of the ACME protocol’s `/.well-known/` directory. An adversary could simply query this known location to determine if a domain owner is actively using the PTX system.

To solve this, PTX employs a technique we term the **anonymous anchor**. The location of the DNS anchor is not static; it is deterministically derived from the secret contents of the proof itself, making it computationally infeasible for an outside observer to discover without possessing the proof artifact.

The process is as follows:

- During proof generation, the prover computes the commitment value, which is the Poseidon hash of the nullifier, secret, and `contextHash`. This value is known only to the prover at this stage.
- The prover then computes a second, public hash (SHA-256) of this commitment field element.
- This SHA-256 hash is encoded into a DNS-safe, 52-character Base32 string. This string serves as the hostname (or “label”) for the TXT record.
- Register the final DNS entry at the hostname `"x-" || base32_hash.example.com`.

To ensure a valid DNS hostname, which cannot begin with a digit according to RFC 952 [14], the final 52-character Base32 string is prepended with a static ASCII character (`'x'`).

This mechanism provides two critical privacy guarantees:

- **Unlinkability**: The Base32 hash string is a pseudorandom value. It has no discernible link to the metadata, the domain, the verifier, or the prover. An observer scanning a domain’s DNS zone would see a set of random-looking TXT records with no context.
- **Undiscoverability**: Without access to the `proof.ptx` file, an attacker cannot compute the correct commitment value and therefore cannot derive the correct hostname to query. The search space of possible 256-bit SHA-256 hashes is computationally infeasible to brute-force.

The act of verification is therefore a secret handshake between the verifier (who can derive the hostname from the file) and the DNS anchor. This makes the verification event itself private and deniable. For an anonymous party, this means they can anchor a proof to their domain without creating a publicly discoverable link between their anonymous platform and the institution they are proving their identity to.

IV. SECURITY ANALYSIS

Having detailed the practical implementation and architecture of the PTX system in the preceding section, we now transition to a formal cryptographic treatment of the protocol. A rigorous security analysis requires that we first define the protocol as a precise set of algorithms and establish the security model in

which it operates. This formalization is essential for constructing sound proofs of its security properties. In this section, we begin by stating our foundational cryptographic assumptions, positioning our analysis within the Random Oracle Model. We then formally specify the PTX protocol as a tuple of three polynomial-time algorithms: *Setup*, *Prove*, and *Verify*. Subsequently, we define the core security properties we aim to achieve, namely existential unforgeability under chosen-message attack (EUF-CMA), replay-resistance, and privacy. Finally, we outline our threat model, defining the capabilities of the adversary against whom these properties must hold. This formalization provides the necessary framework for the rigorous security proofs presented in Section V.

A. Cryptographic Assumptions and the Random Oracle Model

Our security analysis is conducted within the **Random Oracle Model (ROM)** [15]. In this model, cryptographic hash functions are idealized as perfect, publicly accessible random functions. Any party can query an oracle for a given input to receive a uniformly random output, and the only way to determine the output for an input is to query the oracle. This is a standard heuristic for analyzing protocols that use cryptographic hash functions, allowing for clearer and more direct security proofs. We make the following concrete assumptions about our cryptographic building blocks:

- **zk-SNARK Scheme Soundness:** We assume the underlying zk-SNARK scheme, $\Pi = (\text{Setup_snark}, \text{Prove_snark}, \text{Verify_snark})$, instantiated with Groth16, is computationally sound. This property ensures that for any statement x for which no valid witness w exists, it is computationally infeasible for any probabilistic polynomial-time (PPT) adversary to produce a proof σ that is accepted by the *Verify_snark* algorithm, except with negligible probability in the security parameter λ .
- **Collision Resistance of Hash Functions:** The PTX protocol utilizes two hash functions: a ZK-friendly hash H_{zk} (instantiated with Poseidon) for in-circuit commitments, and a public hash H_{pub} (instantiated with SHA-256) for the final metadata commitment. Within the ROM, we assume both H_{zk} and H_{pub} are collision-resistant. This means it is computationally infeasible for any PPT adversary to find two distinct inputs $m_1 \neq m_2$ such that $H(m_1) = H(m_2)$.

These standard assumptions form the basis of our security analysis in Section V. Any adversary capable of breaking the security properties of the PTX protocol can be shown to violate one of these underlying assumptions.

B. Formal Specification of the PTX Protocol

We formally define the PTX protocol for the DNS-based trust anchor (DOH method) as a tuple of three polynomial-time algorithms. Let $\Pi = (\text{Setup_snark}, \text{Prove_snark}, \text{Verify_snark})$ be the underlying zk-SNARK scheme, H_{pub} be a public hash function (e.g., SHA-256), and H_{zk} be a ZK-friendly hash function (e.g., Poseidon).

$\text{Setup}(1^\lambda, C) \rightarrow (\text{pk}, \text{vk})$

The setup algorithm is run once to generate the keys for a specific arithmetic circuit C .

- **Input**
 - 1^λ : Security parameter λ
 - C : A description of the arithmetic circuit *ProofDOH* from Section III.D.
- **Procedure**
 - Run $(\text{pk}, \text{vk}) \leftarrow \text{Setup_snark}(1^\lambda, C)$ to generate the public and private keys for the arithmetic circuit C .
- **Output**
 - pk : The proving key for circuit C .
 - vk : The verification key for circuit C . The verification key is made publicly available as a part of the system's public parameters pp .

$\text{Prove}(\text{pk}, \text{sk}, \text{ctx}) \rightarrow \pi$

The proving algorithm is run by the prover to generate a PTX artifact.

- **Input**
 - pk : The proving key.
 - sk : The prover's secret key, a tuple (nullifier, secret)
 - ctx : The public context for the proof.
- **Procedure**
 - 1) Compute the metadata hash: $\text{metadataHash} \leftarrow H_{\text{pub}}(\text{metadata})$.
 - 2) Split the hash: $(\text{metadataHash_p1}, \text{metadataHash_p2}) \leftarrow \text{Split}(\text{metadataHash})$.
 - 3) Compute the context hash: $\text{contextHash} \leftarrow H_{\text{zk}}(\text{fqdn}, \text{metadataHash_p1}, \text{metadataHash_p2}, \text{trustMethod})$.
 - 4) Compute the nullifier hash: $\text{nullifierHash} \leftarrow H_{\text{zk}}(\text{nullifier})$.
 - 5) Compute final commitment: $\text{commitment} \leftarrow H_{\text{zk}}(\text{nullifier}, \text{secret}, \text{contextHash})$.
 - 6) Assemble public inputs: $x \leftarrow (\text{nullifierHash}, \text{commitment}, \text{fqdn}, \text{metadataHash_p1}, \text{metadataHash_p2}, \text{trustMethod})$.
 - 7) Assemble private inputs (witness): $w \leftarrow (\text{nullifier}, \text{secret})$.
 - 8) Generate the zk-SNARK proof: $\sigma \leftarrow \text{Prove_snark}(\text{pk}, x, w)$.
- **Output**
 - π : The PTX artifact, a tuple $(\sigma, \text{fqdn}, \text{trustMethod}, \text{metadata}, \text{nullifierHash}, \text{commitment})$. This artifact is then serialized into the Protobuf-based file format.
 - **Secret Disposal:** Immediately following the successful generation of σ and its packaging into π , the secret tuple $\text{sk} = (\text{nullifier}, \text{secret})$ is securely discarded. These secrets are treated as “toxic waste”; because the proof system is stateless and non-interactive, there is no requirement for the

Prover to retain any key material once the proof artifact has been produced.

$\text{Verify}(\text{vk}, \text{O_DNS}, \pi) \rightarrow 0, 1$

The verification algorithm is run by the verifier to validate a PTX artifact.

- Input
 - vk : The public verification key.
 - O_DNS : An oracle representing access to the public DNS state. $\text{O_DNS}(\text{hostname})$ returns the value of the corresponding TXT record.
 - π : The PTX artifact to be verified.
 - Procedure
 - 1) **Parse**: Deconstruct π into its components: $(\sigma, \text{fqdn}, \text{trustMethod}, \text{metadata}, \text{nullifierHash}, \text{commitment})$.
 - 2) **Recompute Public Inputs**:
 - a) Compute the expected metadata hash: $\text{expectedMetadataHash} \leftarrow \text{H_pub}(\text{metadata})$.
 - b) Split the hash: $(\text{expectedMetadataHash_p1}, \text{expectedMetadataHash_p2}) \leftarrow \text{Split}(\text{expectedMetadataHash})$.
 - c) Assemble the full public statement x using the values from the parsed π and the recomputed hash parts: $x \leftarrow (\text{nullifierHash}, \text{commitment}, \text{fqdn}, \text{expectedMetadataHash_p1}, \text{expectedMetadataHash_p2}, \text{trustMethod})$.
 - 3) **Anchor Check**:
 - a) Derive anchor hostname: $\text{hostname} \leftarrow \text{"x-"} \parallel \text{Base32}(\text{H_pub}(\text{commitment}))$.
 - b) Query the DNS oracle: $\text{txt_val} \leftarrow \text{O_DNS}(\text{fqdn}, \text{hostname})$.
 - c) Assert equality: If $\text{txt_val} \neq \text{expectedMetadataHash}$, then output 0 and terminate.
 - 4) **Cryptographic Check**:
 - a) Verify the zk-SNARK proof: $\text{Verify_snark}(\text{vk}, x, \sigma)$.
 - b) Assert validity: If $\text{isValid} = \text{false}$, then output 0 and terminate.
 - Output:
 - If all checks pass, output 1 (accept). Otherwise, output 0 (reject).
-

C. Security Definitions

We define the security of the PTX protocol in terms of three primary properties: unforgeability, which ensures the integrity of proofs; replay-resistance, which ensures their correct contextual use; and privacy, which protects the participants' metadata.

1) *Unforgeability (EUF-CMA)*: The foundational security requirement for any credential system is unforgeability. We adopt the standard notion of **Existential Unforgeability under a Chosen Message Attack (EUF-CMA)**. Informally, this property guarantees that no probabilistic polynomial-time (PPT) adversary can create a new, valid PTX proof for a domain that they did not have the authority to issue proofs for, even after observing and requesting legitimate proofs for other domains.

The security game models an adversary A who is given access to a Prove oracle. The adversary can query this oracle on any domain fqdn of its choice to receive a correctly generated PTX proof π . The adversary's goal is to output a new, valid proof π^* for a domain fqdn^* for which it never queried the Prove oracle. The protocol is considered EUF-CMA secure if the probability of any PPT adversary winning this game is negligible in the security parameter.

2) *Replay-Resistance*: Distinct from forgery, a replay attack involves an adversary taking a legitimately generated, valid proof π and fraudulently re-using it in a context for which it was not intended. The PTX protocol must be resistant to two forms of this attack:

- 1) **Cross-Service Replay-Resistance**: This property ensures that a proof π generated for a specific verifier (the audience) cannot be successfully used with a different, unauthorized verifier. An adversary who intercepts a proof intended for verifier A should not be able to present it to verifier B .
 - 2) **Intra-Service Replay-Resistance**: This property ensures that a proof π generated for a specific session or transaction with a verifier cannot be re-used for a different session with the same verifier. An adversary who intercepts a proof for session 1 should not be able to use it to authenticate to session 2.
- 3) *Privacy and Deniability*: The privacy goals of the protocol are twofold, focusing on protecting the metadata of the verification event itself from external observers.

- 1) **Undiscoverability**: This property guarantees that an external adversary without access to the PTX artifact π cannot determine whether a domain owner has created a PTX proof. The adversary should not be able to locate the corresponding DNS anchor record with a probability significantly better than random guessing from the space of all possible hashes.
- 2) **Deniability (Link Severability)**: This property ensures that the public anchor artifact (the DNS record) contains no plaintext information linking the prover, the verifier, or the proof's context. Furthermore, the protocol must provide a mechanism for the prover to retroactively sever this cryptographic link. Once the proof has served its purpose, the prover can delete the DNS record, destroying the public evidence of the verification event and making the proof permanently unverifiable.

D. Threat Model and Adversarial Capabilities

Our security analysis considers a probabilistic polynomial-time (PPT) adversary A whose goal is to undermine the integrity, context, or privacy of the PTX protocol. The adversary

operates in a networked environment and is assumed to have the following capabilities:

- **Network Control:** The adversary A can monitor, intercept, modify, and inject any traffic on the network. This includes observing all communication between the Prover, the DNS infrastructure, and the Verifier. This capability allows the adversary to capture legitimate PTX artifacts π in transit.
- **Public Information Access:** The adversary has full, read-only access to all public information. This includes the public parameters of the system (e.g., the vk), the ability to query the DNS system (including the O_DNS oracle), and access to any other public data sources.
- **Control of Malicious Parties:** The adversary may act as a malicious Prover, a malicious Verifier, or both. They can generate proofs for domains they legitimately control and can attempt to verify any PTX artifact they possess.
- **Oracle Access:** As defined in the EUF-CMA game (Section IV.C.1), the adversary is granted adaptive query access to a `Prove` oracle. This allows them to obtain legitimate PTX proofs for any chosen domain, thereby modeling a "chosen message attack."

Adversarial Goals: The primary goals of the adversary, corresponding to the security properties defined in Section IV.C, are:

- **Forgery:** To create a new, valid PTX artifact π^* for a domain $fqdn^*$ that they do not control and for which they did not obtain a proof from the `Prove` oracle.
- **Replay:** To take a legitimately obtained PTX proof π and have it accepted by a verifier or for a purpose not specified in its purpose metadata.
- **Disclosure:** To discover the existence of a verification event between a Prover and Verifier without prior knowledge of the corresponding PTX artifact.

Assumptions and Limitations: We assume the adversary cannot break the underlying cryptographic primitives defined in Section IV.A (i.e., the soundness of the zk-SNARK and the collision resistance of the hash functions). We also assume the adversary cannot arbitrarily modify DNS records for domains they do not control; this is a standard assumption about the security of the DNS system itself. The integrity of the DNS anchor is further discussed in Section VII.A with respect to DNSSEC.

V. SECURITY ANALYSIS AND FORMAL PROOFS

Having formally defined the PTX protocol, its constituent algorithms, and the adversarial model in the preceding section, we now present the core security analysis of our system. Our analysis is structured to address each of the security goals outlined in Section IV.C. We begin with the most critical property: a formal proof of unforgeability. We demonstrate through a reduction argument that any successful forgery against the PTX protocol implies a break in the computational soundness of the underlying zk-SNARK scheme. Following this, we provide a detailed analysis of the protocol's replay-resistance, focusing on how the cryptographic context-binding of the `audience` and `nonce` fields within the metadata payload mitigates both cross-service and intra-service attacks. Finally, we evaluate the

privacy guarantees of the protocol, examining the "anonymous anchor" mechanism and the "asymmetric disclosure" capability that empowers and protects anonymous provers.

A. Proof of Unforgeability

We now formally prove the primary security guarantee of the PTX protocol: existential unforgeability under a chosen message attack (EUF-CMA). We demonstrate that any adversary capable of forging a PTX proof can be used to construct an algorithm that violates the computational soundness of the underlying zk-SNARK scheme.

1) *The EUF-CMA Security Game:* As defined in Section IV.C.1, the EUF-CMA security is captured by the following game between a challenger C and a probabilistic polynomial-time (PPT) adversary A :

- 1) **Setup:** C runs $Setup(1^\lambda, C)$ to generate the proving key pk and verification key vk . It gives vk (as part of the public parameters pp) to A . C initializes an empty list of queried domains, L , and a DNS oracle, O_DNS .
- 2) **Queries:** A adaptively makes a polynomial number of queries to a `Prove` oracle, simulated by C . For each query on a context $ctx_i = (fqdn_i, metadata_i, \dots)$, C generates a fresh secret key sk_i , runs $Prove(pk, sk_i, ctx_i)$ to get a valid proof artifact π_i , returns π_i to A , adds $fqdn_i$ to the list L , and updates O_DNS to reflect the creation of the corresponding anchor.
- 3) **Forge:** After the query phase, A outputs a forged proof π^* for a context $ctx^* = (fqdn^*, metadata^*, \dots)$.
- 4) **Winning Condition:** A wins if $Verify(vk, O_DNS, \pi^*) = 1$ and $fqdn^* \notin L$.

The advantage of A , denoted $Adv_{PTX, A}^{euf-cma}(\lambda)$, is its probability of winning this game.

2) Reductionist Proof to zk-SNARK Soundness:

Theorem 1. Let $\Pi = (Setup_snark, Prove_snark, Verify_snark)$ be a computationally sound zk-SNARK scheme, and let H_{pub}, H_{zk} be hash functions modeled as random oracles. Then the PTX protocol is existentially unforgeable under a chosen message attack (EUF-CMA secure).

Proof. We prove by reduction. Assume for contradiction that there exists a PPT adversary A that breaks the EUF-CMA security of the PTX protocol with a non-negligible probability $\epsilon(\lambda)$. We will construct a PPT algorithm B that uses A as a subroutine to break the computational soundness of the zk-SNARK scheme Π with non-negligible probability.

The soundness game for Π requires B to produce a valid proof σ^* for a public statement x for which B does not know a corresponding witness w . B interacts with a SNARK challenger, C_{snark} , as follows:

- 1) **Setup:** C_{snark} generates the SNARK keys (pk, vk) for the circuit C and gives them to B . B then starts an instance of the adversary A . B gives the verification key vk (as part of the PTX public parameters) to A . B will now simulate the entire environment for A , including the EUF-CMA challenger and the Random Oracles. To

do so, B maintains lists for the H_{pub} and H_{zk} oracles, as well as the list of queried domains L and the DNS oracle O_{DNS} .

2) Simulation of Oracles:

- **Random Oracles (H_{pub}, H_{zk}):** When A queries an oracle with an input m that B has not seen before, B generates a random value r , stores the pair (m, r) in the corresponding list, and returns r . If m has been queried before, B returns the stored r .
- **Prove Oracle:** When A makes a *Prove* oracle query on a context ctx_i , B plays the role of the challenger perfectly. B generates a fresh secret key $sk_i = (nullifier_i, secret_i)$, computes the full witness w_i and public statement x_i by querying its simulated oracles. Since B knows both x_i and w_i , it can use the real proving key pk to run $Prove_snark(pk, x_i, w_i)$ and generate a valid SNARK proof σ_i . B constructs the full PTX artifact π_i , returns it to A , and updates L and its simulated O_{DNS} oracle as required by the game.

3) **Extraction from Forge:** By our initial assumption, A eventually outputs a forged proof π^* for a fresh domain $fqdn^*$ with non-negligible probability ϵ . For A to win, $Verify(vk, O_{DNS}, \pi^*)$ must equal 1.

4) **Analysis of the Forgery:** For $Verify(vk, O_{DNS}, \pi^*)$ to return 1, two deterministic checks must pass:

- a) **Anchor Check:** The DNS query $O_{DNS}(fqdn^*, hostname^*)$ must return $expected_metadataHash^*$.
- b) **ZK Check:** $Verify_snark(vk, x^*, \sigma^*)$ must return true.

Since $fqdn^* \notin L$, B never programmed the O_{DNS} oracle for this domain. Therefore, for the Anchor Check to pass, the adversary A must have made the necessary Random Oracle queries to compute both $hostname^*$ and $expected_metadataHash^*$ themselves. The values $hostname^*$ and $commitment^*$ (from which $hostname^*$ is derived) depend on the witness w^* .

5) **Winning the Soundness Game:** B parses A 's forgery π^* to extract the SNARK proof σ^* and the public statement x^* . Since the ZK Check (b) must have passed for A to win, B knows that the pair (x^*, σ^*) is a valid statement-proof pair for the SNARK scheme. B now presents this pair to its own challenger, C_{snark} .

For B to win its game, it must not know the witness $w^* = (nullifier^*, secret^*)$ corresponding to x^* . Let us analyze how A (and by extension, B) could possibly come to know w^* .

The public statement x^* contains the values $nullifierHash^*$ and $commitment^*$. These are defined as $H_{zk}(nullifier^*)$ and $H_{zk}(nullifier^*, secret^*, contextHash^*)$ respectively. For A to know the witness w^* , it must have found values $(nullifier^*, secret^*)$ that produce these outputs when hashed. This would require A to find a pre-image for the outputs of the random oracle H_{zk} . In the ROM, this is computationally infeasible for a PPT algorithm except with negligible probability.

Since ctx^* is for a fresh domain, A was never provided a witness for this context by the Prove oracle. Therefore, A 's only path to knowing w^* is by inverting the random oracle. In the ROM, the probability of a PPT adversary finding a pre-image for a random oracle's output is negligible.

Thus, A must have produced a valid SNARK proof σ^* for a public statement x^* without knowledge of the corresponding witness w^* .

The algorithm B has successfully used the adversary A to produce a valid pair (x^*, σ^*) that breaks the computational soundness of the zk-SNARK scheme Π . The probability of B winning its game is at least the probability of A winning its game, minus the negligible probability of A successfully inverting the random oracle: $Adv_{\Pi, B}^{sound}(\lambda) \geq \epsilon(\lambda) - negl(\lambda)$.

Since we assumed Π is computationally sound, $Adv_{\Pi, B}^{sound}(\lambda)$ must be negligible. This implies that $\epsilon(\lambda)$ must also be negligible. This is a contradiction to our initial assumption that a successful adversary A exists with non-negligible advantage.

Therefore, no such adversary A can exist, and the PTX protocol is EUF-CMA secure. $\square$

B. Formal Analysis of Replay-Resistance

While unforgeability guarantees that an adversary cannot create a new valid proof, it does not prevent them from maliciously re-using a legitimately generated proof. A replay attack occurs when an adversary intercepts a valid proof artifact π and presents it in a context for which it was not intended. The PTX protocol's security against such attacks is not derived from the core zk-SNARK soundness property, but from the strict, verifiable context-binding enforced by the SignedMetadataPayload and validated by the verifier's security policy. We analyze the two primary forms of this attack.

Mitigating Cross-Service Replay via the audience Binding Parameter

Attack Scenario: An adversary A wishes to impersonate a prover P to a target verifier, $Verifier_B$ (e.g., an enterprise service). A observes P generating a legitimate proof π for a different, honest verifier, $Verifier_A$ (e.g., a university portal). A intercepts π and submits it to $Verifier_B$.

Defense Mechanism: The audience field within the purpose object of the SignedMetadataPayload is the primary defense against this attack.

Analysis:

The legitimate proof π was generated with a purpose object explicitly stating audience: "verifier_A.com". This string is an integral part of the metadata payload that is hashed to produce the `metadataHash_p1` and `metadataHash_p2` public inputs for the zk-SNARK. The cryptographic proof σ contained within π is therefore mathematically valid *only* for a statement that includes audience: "verifier_A.com".

A correctly implemented verifier, $Verifier_B$, must enforce a strict security policy as part of its *Verify* algorithm. Before proceeding to the cryptographic checks, it must first parse the metadata from the submitted proof π and assert that

its own identity (e.g., "verifier_B.com") is present in the audience field.

When Verifier_B receives the replayed proof π , it will inspect the audience field and find "verifier_A.com". Since this does not match its own identity, the verification process terminates immediately and rejects the proof. This check is a semantic, application-level guardrail that occurs before any expensive cryptographic operations. The proof is rejected not because the mathematics is invalid in a vacuum, but because the proof is semantically irrelevant and unauthorized for the current context.

We formally prove that the PTX protocol is resistant to both cross-service and intra-service replay attacks. This is achieved by defining two security games and showing that the probability of any PPT adversary winning either game is negligible, assuming a correctly implemented verifier.

1) *Security Game: Cross-Service Replay-Resistance (CS-RR)*: This game formalizes the intuition that a proof intended for one audience cannot be used at another.

Definition: The PTX protocol is Cross-Service Replay-Resistant if for any PPT adversary A , its advantage $Adv_{PTX,A}^{cs-rr}(\lambda)$ in the following game is negligible.

The CS-RR Game:

- 1) *Setup*: The challenger C runs $Setup(1^\lambda, C)$ to generate keys (pk, vk) and gives vk (as part of pp) to the adversary A . C initializes the Prove oracle and the DNS oracle O_{DNS} .
- 2) *Queries*: A can make a polynomial number of queries to the Prove oracle on any context ctx_i of its choosing. C responds with a valid proof π_i .
- 3) *Challenge*: A chooses two distinct, valid audience identifiers, aud_A and aud_B . A then chooses a challenge context ctx^* which includes aud_A . A submits ctx^* to the Prove oracle.
- 4) *Response*: C generates a fresh secret key sk^* , runs $Prove(pk, sk^*, ctx^*)$ to generate the challenge proof π^* , updates O_{DNS} accordingly, and returns π^* to A .
- 5) *Replay*: A must now attempt to get the exact same proof artifact π^* accepted by a verifier who identifies as aud_B . This is modeled by A submitting π^* to a Verify oracle simulated by C , where this oracle assumes the identity of aud_B .
- 6) *Winning Condition*: A wins if the check $Verify(vk, O_{DNS}, \pi^*, aud_B)$ returns 1.

Theorem 2. *The PTX protocol is Cross-Service Replay-Resistant, provided that the verifier correctly implements the Verify algorithm as specified in Section IV.B.*

Proof. We prove that the adversary's advantage is zero.

- 1) Let A be a PPT adversary playing the CS-RR game. A receives the challenge proof π^* which was generated for the context ctx^* where `purpose.audience` is aud_A .
- 2) By the definition of the Prove algorithm, the `signed_metadata` string within π^* contains the JSON substring "audience": " aud_A ".

- 3) The adversary A wins if $Verify(vk, O_{DNS}, \pi^*, aud_B) = 1$.
- 4) Let us analyze the execution of the *Verify* algorithm when called with the verifier identity aud_B . According to our specification, the *Verify* algorithm is a deterministic procedure. A compliant verifier must perform a semantic check before any cryptographic operations.
- 5) This semantic check involves parsing the `signed_metadata` from π^* and asserting that the verifier's own identity (aud_B) is contained within the audience field of the metadata.
- 6) The verifier will extract aud_A from the metadata. The check becomes: `is  $aud_B$  in  $\{aud_A\}$ ?`
- 7) By the game's definition, $aud_A \neq aud_B$. Therefore, the check will evaluate to false.
- 8) Upon this failure, the Verify algorithm is specified to terminate immediately and return 0.
- 9) This contradicts the winning condition, which requires the algorithm to return 1.

□

Therefore, no adversary A can win the CS-RR game, and the protocol is secure against cross-service replay attacks. The security is not cryptographic but procedural, relying on the mandatory and deterministic policy enforcement by the verifier.

2) *Security Game: Intra-Service Replay-Resistance (IS-RR)*: This game formalizes the property that a proof is a single-use, transactional credential.

Definition: The PTX protocol is Intra-Service Replay-Resistant if for any PPT adversary A , its advantage $Adv_{PTX,A}^{is-rr}(\lambda)$ in the game below is negligible.

The IS-RR Game:

- 1) *Setup*: The challenger C runs $Setup(1^\lambda, C)$ to generate keys (pk, vk) and gives vk to the adversary A . C initializes the *Prove* oracle, the DNS oracle O_{DNS} , and an empty set N_{used} which will store all nonces that have been successfully used. C will also simulate a *Verify* oracle.
- 2) *Transaction 1 (Legitimate Use)*:
 - a) A initiates a transaction with C .
 - b) C generates a fresh, random nonce n_1 from a sufficiently large space and gives it to A .
 - c) A chooses a context ctx_1 which must include nonce: n_1 .
 - d) A submits ctx_1 to the Prove oracle. C generates the proof π_1 and returns it.
 - e) A submits π_1 to the Verify oracle. C verifies it. Since it is valid and n_1 is correct, the oracle returns 1 and C adds n_1 to the set N_{used} .
- 3) *Transaction 2 (Replay Attempt)*:
 - a) A initiates a new transaction with C .
 - b) C generates a new, fresh, random nonce n_2 (where $n_2 \neq n_1$) and gives it to A .
 - c) A must now attempt to replay the proof from the first transaction. A submits the original proof π_1 to the Verify oracle for this new transaction.

- 4) *Winning Condition*: A wins the game if the Verify oracle returns 1 for the submission of π_1 in Transaction 2.

Theorem 3. *The PTX protocol is Intra-Service Replay-Resistant, provided that the verifier correctly implements the Verify algorithm as specified and maintains a stateful record of used nonces.*

Proof. We prove that the adversary’s advantage is zero.

- 1) Let A be a PPT adversary playing the IS-RR game. A possesses the valid proof π_1 , which was generated for the context ctx_1 containing nonce n_1 .
- 2) A wins if the Verify oracle, when called in the context of Transaction 2 (which expects nonce n_2), returns 1 for the proof π_1 .
- 3) Let us analyze the execution of the Verify algorithm during Transaction 2. A compliant, stateful verifier must perform two semantic checks related to the nonce before cryptographic operations:
 - a) Match Check: Does the nonce in the proof match the nonce expected for the current transaction?
 - b) Uniqueness Check: Has this nonce ever been used in a successful verification before?
- 4) The verifier parses the `signed_metadata` from the replayed proof π_1 and extracts the nonce, which is n_1 .
- 5) The verifier then performs the Match Check. It compares the nonce from the proof (n_1) with the nonce it generated for Transaction 2 (n_2).
- 6) By the game’s definition, $n_1 \neq n_2$. Therefore, the Match Check evaluates to `false`.
- 7) The Verify algorithm is specified to terminate immediately and return 0.
- 8) This contradicts the winning condition, which requires the algorithm to return 1.

Alternative Scenario: Let us consider a flawed verifier that does not perform a Match Check but only a Uniqueness Check. In Transaction 1, the nonce n_1 was added to the set N_{used} . When A replays π_1 , the verifier extracts n_1 and checks if $n_1 \in N_{used}$. Since this is true, the Uniqueness Check fails, and the algorithm returns 0.

In either case, the adversary cannot win. The protocol is therefore secure against intra-service replay attacks, relying on the mandatory, deterministic, and stateful enforcement of nonce validation by the verifier. $\square$

C. Analysis of Privacy and Deniability

Beyond the core integrity guarantees of unforgeability and replay-resistance, the PTX protocol is explicitly designed to provide strong privacy protections for the prover. This analysis focuses on two key properties: the protocol’s ability to conceal the existence of a verification event from outside observers, and its mechanism for enabling selective, time-limited disclosure of identity.

1) The “Anonymous Anchor” Mechanism and Undiscoverability:

Attack Scenario: An external adversary A , without access to any PTX artifacts, wishes to determine if the owner of `example.com` is actively using the PTX protocol. A naive anchoring scheme, such as placing a record at a well-known location (e.g., `_ptx-proof.example.com`), would make this trivial, creating a publicly observable side channel for metadata leakage.

Defense Mechanism: The PTX protocol defends against this by making the anchor’s location computationally undiscoverable without possession of the proof artifact itself.

Analysis: The DNS hostname for the anchor is not fixed; it is deterministically derived from the proof’s commitment: `hostname  $\leftarrow$  ("x-" || Base32(H_pub(commitment)))`. The commitment is the output of `H_zk(nullifier, secret, contextHash)`.

For an adversary A to discover the correct hostname to query, they would need to find the output of this hash chain. Since `H_pub` and `H_zk` are modeled as random oracles, and the inputs `nullifier` and `secret` are private to the prover, the adversary is faced with a search problem. The space of possible hostname values is computationally equivalent to the output space of the hash function (e.g., 2^{256} for SHA-256).

It is therefore computationally infeasible for an adversary to scan, guess, or brute-force the correct hostname. The verification event is invisible to any party not in possession of the π artifact. This “anonymous anchor” mechanism ensures that the act of creating a proof does not, in itself, create a publicly discoverable footprint.

2) Asymmetric Disclosure and Link Severability:

Attack Scenario: A prover P (e.g., the anonymous operator of a political blog) needs to prove their ownership of their domain to a specific, trusted verifier V (e.g., a university admissions office) without publicly revealing their real-world identity to their potentially hostile general audience A .

Defense Mechanism: The combination of private proof transmission, the “anonymous anchor,” and DNS-based revocation enables a workflow of selective, asymmetric disclosure.

Analysis:

The protocol flow for this scenario is as follows:

- 1) **Private Generation:** P generates a PTX proof π with the audience field set specifically to V .
- 2) **Private Transmission:** P transmits π to V over a private channel (e.g., a secure application portal).
- 3) **Private Verification:** Only V , who now possesses π , can derive the correct hostname and perform the verification. The general audience A , who does not have π , cannot discover the anchor and remains unaware of the event. The disclosure of identity is therefore asymmetric: V learns the identity, but A does not.
- 4) **Link Severability:** After V confirms the verification, P can delete the TXT record from their DNS zone. The cryptographic link between the public domain and the private proof artifact π is now permanently severed. The proof π becomes unverifiable by anyone, including V , after the DNS Time-to-Live (TTL) expires.

This provides a powerful “right to be forgotten” for cryptographic credentials. The PTX anchor’s existence is temporary, its location is private to the participants, and its removal destroys all public evidence of the event. This makes the protocol suitable for high-stakes verification scenarios where long-term, public linkability is a direct threat to the prover.

D. Revocation Latency vs. Certificate Revocation Lists (CRLs)

A critical security feature of any credentialing system is not just its issuance, but its ability to be reliably and promptly revoked. The PTX protocol provides a simple, direct, and highly effective revocation mechanism by leveraging the inherent properties of the Domain Name System.

1) *The PTX Revocation Model:* To revoke a PTX proof, or all proofs associated with a specific anchor, the prover can revoke the proof’s validity globally by simply *deleting the corresponding TXT record from their DNS zone*. The propagation of this revocation is governed by the record’s Time-to-Live (TTL) value, which can be configured to be very short (e.g., 60-300 seconds). Once the TTL expires from upstream DNS caches, any subsequent verification attempt for a PTX artifact pointing to that anchor will fail the Anchor Check (Section IV.B), as the `O_DNS` oracle will return `NXDOMAIN` (non-existent domain). This revocation is a single, $O(\text{TTL})$ operation on the prover’s side, and its effect is guaranteed and global.

2) *Comparison with X.509 Revocation:* This stands in stark contrast to the cumbersome and often ineffective revocation mechanisms within the traditional X.509 PKI. The two primary methods are:

- 1) **Certificate Revocation Lists (CRLs):** CRLs are periodic lists of revoked certificates that are distributed to clients. The process of revoking a certificate is complex and requires coordination between the Certificate Authority (CA) and the client. CRLs are not always up-to-date, and their distribution can be slow and unreliable.
- 2) **Online Certificate Status Protocol (OCSP):** OCSP is an online protocol that allows clients to check the status of a certificate in real-time. However, OCSP servers are often centralized and can be subject to Denial of Service (DoS) attacks. OCSP stapling, a variant of OCSP, can help mitigate these issues, but it is not always supported by clients or servers. Furthermore, OCSP verification has a potential to leak data to the server, which can be used for tracking and profiling.

The PTX protocol’s DNS-based approach avoids these pitfalls entirely. It requires no new infrastructure, has no centralized chokepoint for status checks, and preserves privacy, as the verifier’s query to a public DNS resolver does not reveal its identity to the original issuer. This offers a superior model for fast and verifiable credential invalidation (ideally in $O(\text{TTL})$ time).

VI. PERFORMANCE EVALUATION

Having formally established the security properties of the PTX protocol, we now turn to its practical viability. A secure but computationally prohibitive protocol is of little use in

real-world systems. This section therefore presents a comprehensive empirical evaluation of our reference implementation to quantify its performance characteristics across a range of representative computing environments. The objective of this evaluation is to measure the computational and network costs associated with the protocol’s entire lifecycle and to identify any potential performance bottlenecks.

Our analysis is structured around four key areas of performance: (1) the static complexity of the `ProofDoH` arithmetic circuit and the resulting key material sizes; (2) the computational cost for the prover, including a parametric analysis of how performance scales with public input size; (3) the end-to-end latency for the verifier, carefully distinguishing between network-bound I/O and CPU-bound cryptographic operations; and (4) the storage and transmission cost of the final PTX artifact. To provide a comprehensive performance profile, all benchmarks were conducted across a heterogeneous set of hardware platforms—from server-class CPUs to mobile devices—utilizing both a native Go implementation (`gnark` [16]) and a WebAssembly-based library (`websnark` [17]) to capture the performance trade-offs between native and web-based execution environments.

A. Circuit and Key Material Metrics

The first step in evaluating the performance of the PTX protocol is to establish the baseline computational and storage costs inherent to the `ProofDoH` circuit itself. These static metrics dictate the fixed overhead for any implementation of the system.

The compiled R1CS representation of our `ProofDoH` circuit contains a total of 1,756 constraints. This low constraint count, achieved primarily through the use of the ZK-friendly Poseidon hash function (as discussed in Section II.C), is the primary determinant of the prover’s computational workload and the size of the trusted setup parameters. The circuit is defined over 2 private inputs (`nullifier`, `secret`) and 6 public inputs.

The trusted setup for this circuit yields two critical cryptographic artifacts: the proving key and the verification key. Their sizes are summarized below:

- *Proving Key Size:* The final proving key (`.zkey` file) required by the prover to generate proofs has a size of 765,454 bytes (~748 KiB).
- *Verification Key Size:* The verification key (`.vkey` file) required by the verifier is substantially smaller, with a size of just 4,722 bytes (~4.6 KiB).

This significant asymmetry in key size is a key feature of the Groth16 scheme and has important practical implications. The larger proving key is a one-time asset for a prover’s client-side application or a server-side generation service. In contrast, the extremely small verification key can be efficiently cached by verifiers, embedded directly within lightweight applications, or served to web clients with minimal network overhead. These static metrics form the fixed foundation upon which the dynamic runtime performance, evaluated in the following sections, is built.

B. Comparative Analysis of ZKP Schemes

To situate the performance of the PTX protocol within the broader landscape of zero-knowledge cryptography, Table I provides a comparative analysis of our chosen scheme, Groth16, against other prominent proof systems: Halo2 [18], Bulletproofs [19], and STARKs [20].

TABLE I: Comparative Benchmarks of ZKP Proving Systems (approximated for equivalent circuit complexity)

Scheme	Proof Size	Verification	Trusted Setup
Groth16	~650 B	~1.5 ms	Per-Circuit
Halo2 (KZG)	~400 B	~2.5 ms	Universal
Bulletproofs	~1.5 KiB	~100+ ms	None
STARKs	~45 KiB	~5.0 ms	None

This comparison highlights the primary trade-off governing the design of the PTX protocol. While Halo2 and STARKs offer superior “transparency” by eliminating the need for a per-circuit trusted setup, they do so at the cost of significantly larger proof sizes and, in the case of STARKs, a memory-intensive proving phase that is less suitable for client-side execution in resource-constrained environments. For the PTX protocol, where the “portability” of the proof artifact (e.g., embedding it in a DNS record or a lightweight JSON payload) and the speed of verifier response are paramount, Groth16 remains the optimal choice despite its setup requirements.

C. Prover Performance: A Parametric Sweep of Public Inputs

While the overall constraint count dictates the baseline complexity of the prover’s task, it is also critical to evaluate how performance is affected by the size of the inputs themselves. A significant performance degradation when processing larger public inputs could limit the protocol’s practical applicability. To investigate this, we conducted a parametric sweep of the `fqdn` public input, as its variable length represents the most likely source of performance variance.

The experiment consisted of repeatedly running the proof generation algorithm while varying the length of the `fqdn` string from a minimum of 3 bytes to the maximum-allowed 255 bytes. Benchmarks were conducted on two distinct hardware profiles: a modern consumer laptop under active use (Apple M4) to simulate a real-world client-side scenario, and an idle cloud server instance (Intel Xeon) to establish a baseline in a controlled environment. The results are depicted in Figures 3 and 4.

The empirical data reveals two key findings. First, as shown in both figures, the median and mean proof generation times remain remarkably constant across the entire range of FQDN lengths. This result empirically validates the theoretical expectation that the prover’s computational complexity is a function of the circuit’s fixed constraint count, not the bit-length of the public inputs. The computational cost is effectively $O(1)$ with respect to the input values’ size.

Second, the performance is well within the bounds of practical usability. On the consumer-grade M4 hardware (Figure 3), the mean proof generation time remained consistently below 50 ms, far below the commonly accepted 500 ms threshold

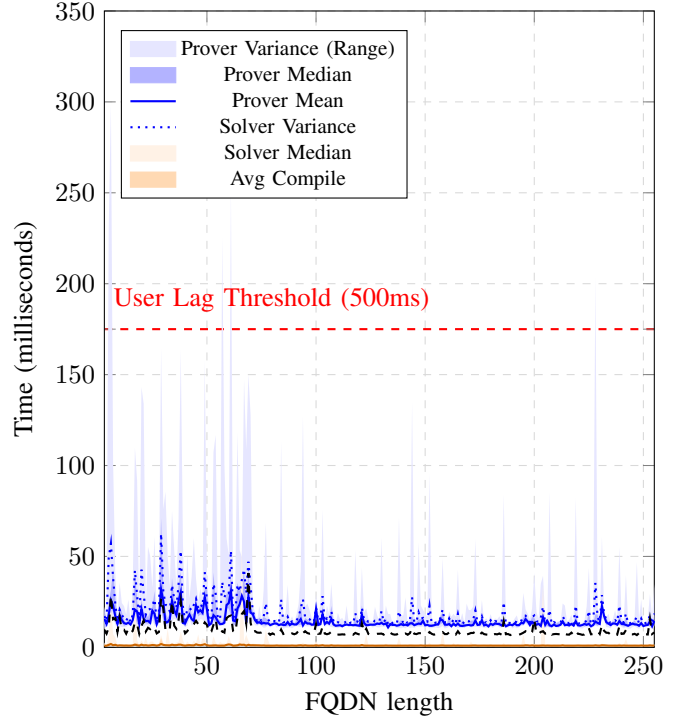


Fig. 3: Performance breakdown during parametric sweep of the FQDN variable from 3 to 255 bytes reveals a negligible impact on proof time on active consumer hardware (MacBook Air M4, 2025).

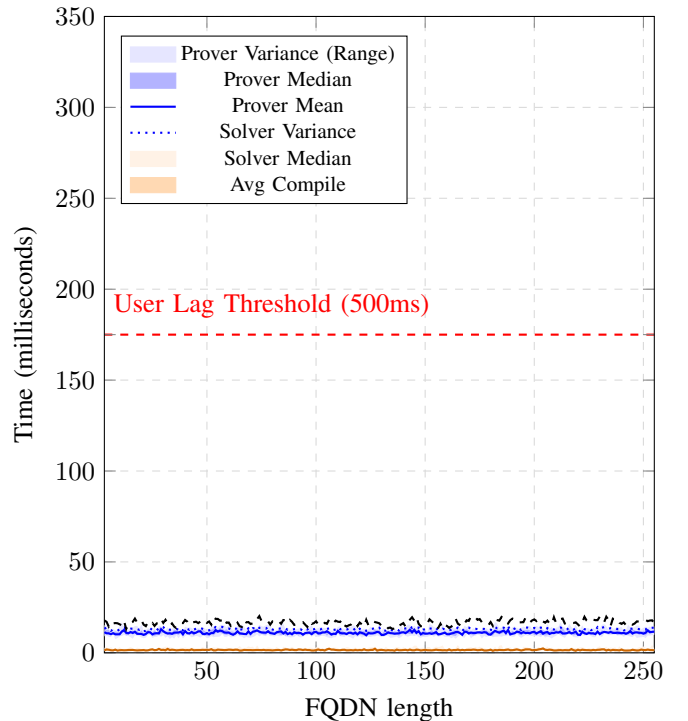


Fig. 4: Performance breakdown during parametric sweep of the FQDN variable from 3 to 255 bytes reveals a negligible impact on proof time on stable, idle server hardware (Intel Xeon Gold, Ubuntu 24.04).

for perceived user lag. This indicates the viability of on-device, client-side proof generation for interactive applications. The higher variance observed on the actively used consumer hardware compared to the idle server (Figure 4) is attributed to non-deterministic operating system task scheduling, a factor further analyzed in Section VI.G. This analysis demonstrates that the prover’s performance is scalable and predictable, showing no degradation even when processing maximum-length public inputs.

D. Verifier Performance: Isolating Cryptographic and Network Latency

The total time required for a verifier to validate a PTX proof is a composite of two distinct and independent operations: the network-bound I/O for the DNS anchor check, and the CPU-bound computation for the cryptographic proof verification. To accurately characterize the performance and identify the primary bottleneck, we designed an experiment to isolate and measure these two components.

We implemented our native Go verifier with two operational modes: a ‘Full Verification’ mode that performs all steps of the protocol, and a ‘ZK-Only’ mode that bypasses the DNS query to measure only the raw cryptographic overhead. We conducted 250 test runs in each mode on an idle cloud server instance (Intel Xeon). The aggregated results, shown in Table II, provide a clear breakdown of the verification latency.

TABLE II: Breakdown of Verification Latency (Intel Xeon, Idle)

Component	Average	Std. Dev.
DNS Fetch	52.15 ms	7.30 ms
Proof Verification	1.665 ms	0.541 ms
Total Time	53.815 ms	1.545 ms

The data reveals a stark performance disparity. The cryptographic verification of the Groth16 proof is exceptionally fast, averaging just **1.665 milliseconds**. In contrast, the DNS query to fetch the anchor record averaged **52.15 milliseconds**, over an order of magnitude slower.

The analysis is further illustrated by the stacked bar chart in Figure 5, which visually represents the latency breakdown. The total verification time is overwhelmingly dominated by the network latency of the DNS fetch. Furthermore, the standard deviation of the total time (7.29 ms) is almost entirely attributable to the variance in network performance (7.30 ms), while the cryptographic component remains highly stable and deterministic (0.541 ms).

This empirical result is a key finding of our work: the performance of the PTX verifier is not bound by the complexity of the zero-knowledge cryptography, but rather by the speed and caching state of the public internet’s DNS infrastructure. This indicates that for latency-critical applications, any significant performance optimization efforts should be directed towards DNS resolution strategies rather than the cryptographic pipeline itself.

E. Cross-Platform Benchmark Analysis

To assess the practical viability of the PTX protocol across a spectrum of real-world computing environments, we conducted

a comprehensive benchmark across four representative hardware platforms: a modern consumer laptop (Apple M4), an older cloud server instance (Intel Xeon G-6138), a mainstream mobile device (Snapdragon 732G), and a premium tablet (Apple iPad). For each platform, we measured the performance of both our native Go implementation (gnark [16]) and our WebAssembly-based library (websnark [17] via snarkjs [21]) where applicable. Each test configuration was run 1,000 times to obtain stable median performance metrics for both proof generation and cryptographic verification.

First, the most immediate observation from Table III is the significant performance advantage of the native Go implementation over the browser-based WASM library. On the same Apple M4 hardware, the median proof generation time increases by over 15x (17.280ms vs. 258.70ms) when moving from native execution to the browser sandbox. A similar, though more pronounced, disparity exists for verification. This “browser tax” is a critical finding and will be analyzed in detail in Section VI.F.

Second, the data empirically validates the fundamental asymmetry of the zk-SNARK scheme: proving is computationally more expensive than verification across all tested configurations. This asymmetry is most pronounced in the native Go implementation on the M4, where verification is approximately 14.5 times faster than proof generation. This property is highly desirable for the PTX use case, as proofs are typically generated once by a single prover but may be verified many times by multiple verifiers.

Crucially, the performance on mobile devices remains within the bounds of practical usability. On the mainstream Snapdragon 732G device, proof generation is achievable in under a second (721.75 ms), while verification is performed in just 155.64 ms. This is well below the threshold for interactive user applications, confirming the viability of PTX for the mobile-first web. The benchmarks also reveal nuanced performance characteristics, particularly the surprisingly high performance of the iPad tablet, which will be examined in Section VI.G.

F. The “Browser Tax”: Quantifying the JIT vs. AOT Performance Gap

One of the most significant findings from our cross-platform analysis is the substantial performance overhead incurred when executing ZK-SNARK operations within a web browser’s JavaScript and WebAssembly (WASM) environment. We term this overhead the “browser tax.” This effect is most clearly isolated by comparing the results on the Apple M4 platform, where the same underlying hardware was used for both the native, Ahead-of-Time (AOT) compiled Golang binary and the web-based WASM benchmark running in a Just-in-Time (JIT) compiled JavaScript engine.

As shown in Table III, the median proof generation time on the Apple M4 increased from **17.280 ms** for the native binary to **258.70 ms** for the WASM implementation—a slowdown factor of approximately **15x**. A similar, though less pronounced, effect was observed for the verification task. This performance gap is not an artifact of the PTX protocol itself but is indicative of the

Latency Component Comparison by Environment

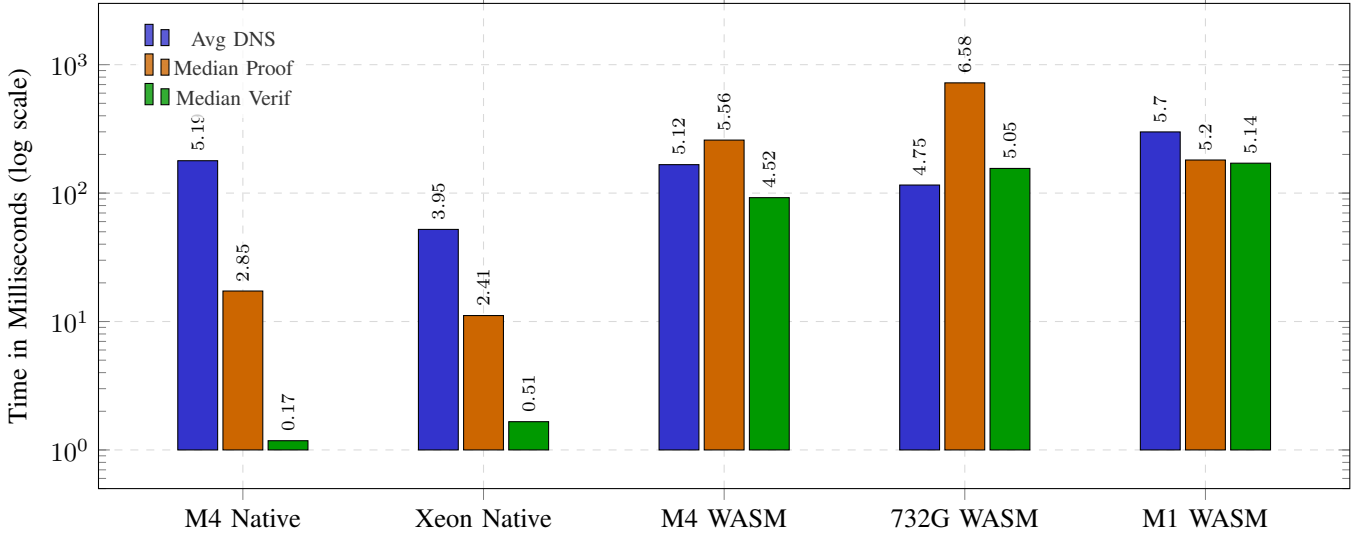


Fig. 5: Component-wise latency breakdown. Native Go binaries exhibit superior cryptographic performance (proof/verification) compared to WASM, while DNS resolution remains the primary bottleneck for low-latency native execution.

TABLE III: **Key Statistics Across Test Environments.** Performance metrics observed across the five most common consumer configurations.

Environment	CPU	Load Type	Median Proof	Median Verif.	Rationale
Golang binary	Apple M4	Active	17.280 ms	1.189 ms	Modern edge consumer device.
KVM, Golang binary	Xeon G-6138	Idle	52.151 ms	1.666 ms	Older datacenter server (2017).
webSNARK, WASM (snarkjs)	Apple M4	Active	258.70 ms	92.160 ms	Browser-based execution.
webSNARK, WASM (snarkjs)	Snapdragon 732G	Active	721.75 ms	155.64 ms	Mainstream mobile device.
webSNARK, WASM (snarkjs)	M1 (iPad)	Active	181.00 ms	171.00 ms	Modern mobile tablet.

fundamental architectural differences between native and web-based execution models for this specific, highly-specialized workload.

We attribute this performance delta to several factors inherent to the web execution model:

- *The JavaScript-to-WASM Boundary:* While WASM allows for near-native execution speed for the core cryptographic computations, `snarkjs` requires frequent data marshalling across the boundary between the JavaScript runtime and the WASM module. This boundary crossing is not a zero-cost abstraction, and the cumulative overhead of these calls contributes significantly to the total latency.
- *Large Integer Arithmetic:* The core of all zk-SNARK operations involves multi-precision arithmetic over large prime fields. Native cryptographic libraries are often written in low-level languages and can leverage highly-optimized, assembly-level instructions for these operations. In contrast, JavaScript’s `BigInt` and the corresponding WASM implementations, while functional, are typically less specialized and may not fully exploit the underlying CPU’s capabilities for this type of math.
- *The Browser Sandbox:* All code executed within a browser runs inside a strict security sandbox. This essential security feature introduces a layer of abstraction and virtualization that, while necessary, incurs an inherent performance cost compared to a native binary that interacts more directly with the operating system and hardware.

This finding has significant implications for the practical deployment of client-side ZK applications. While our results confirm that browser-based proving and verification are viable (as seen in Section VI.E), developers of latency-sensitive applications must be cognizant of this “browser tax.” For applications requiring the lowest possible latency, a hybrid model where computationally expensive proof generation is offloaded to a native server-side component may be a more suitable architecture.

G. Performance Variance and OS Jitter in Mainstream Mobile Environments

While median performance provides a baseline, the *consistency* of that performance is a critical factor for applications aiming to provide a responsive user experience on mainstream mobile devices. Our detailed benchmarks on a device equipped with a Qualcomm Snapdragon 732G System-on-Chip—a processor representative of the broad consumer Android market—revealed significant performance variance, highlighting the challenges of deploying deterministic cryptographic operations in non-deterministic, consumer-grade environments.

The aggregated statistics from 1,000 runs of the proof generation and verification tasks on the Snapdragon device (Chrome, WASM) are presented below:

We observe a notable divergence between the median (721.75 ms) and the average (790 ms). The average is being skewed upwards by high-latency outliers, with the maximum observed

time being nearly double the median. This high standard deviation indicates substantial performance jitter.

We attribute this variance to the inherent characteristics of a mainstream mobile operating system like Android. The OS scheduler is aggressively multitasking, managing background services, network connectivity, and UI events. The proof generation process, being a computationally intensive, long-running task, is frequently preempted by these higher-priority operations. This effect is compounded by events within the JavaScript engine itself, such as garbage collection cycles, which can introduce unpredictable pauses.

This finding has crucial implications for the design of client-side ZK systems targeting the broad mobile market. While our data confirms that sub-second proof generation is viable on mainstream hardware, developers must architect their applications to be resilient to this significant, inherent performance jitter. Applications requiring a smooth user experience must not assume deterministic, real-time performance. Instead, they should employ asynchronous patterns, such as performing cryptographic operations in background service workers and utilizing loading indicators, to avoid blocking the main UI thread and gracefully handle the unpredictable latency of consumer mobile environments.

H. JIT Compiler Warm-up and Cold Start Latency in WASM Runtimes

A nuanced aspect of performance, particularly for web-based applications, is the “cold start” latency associated with Just-in-Time (JIT) compilation. Our extensive benchmarks of the websnark implementation revealed a consistent and significant performance penalty during the initial executions of the proof generation function, a phenomenon we attribute to the warm-up phase of the browser’s JavaScript and WebAssembly engine.

consistently demonstrated latencies in the 1,000-1,100ms range, approximately 40-50% slower than the stable median of 721ms that was achieved in subsequent runs. A similar, though less pronounced, pattern was observed across all tested browser-based environments. This initial overhead occurred despite all cryptographic key material and WASM modules being pre-loaded into memory, confirming that the bottleneck is computational rather than network-bound.

We attribute this behavior to the operational model of modern JIT compilers like V8 [22]. Upon the first invocation of a function, the engine begins by running the code through a baseline interpreter, which allows for immediate execution at the cost of performance. Concurrently, a profiler monitors the code, identifying “hot spots”—computationally intensive loops and functions—such as those found in the `groth16.fullProve` algorithm. Once identified, these hot spots are passed to an optimizing compiler (e.g., TurboFan in V8), which recompiles the WASM bytecode into highly-optimized, platform-specific machine code. This optimization process is itself resource-intensive and occurs in the background. After a few executions, the engine “hot-swaps” the slow, interpreted version with the newly compiled native code, at which point the performance stabilizes at its optimal, lower latency.

This “JIT warm-up” has critical implications for the user experience of web-based ZK applications. A user’s very first interaction with a proving function may be noticeably slower than all subsequent interactions. To mitigate this initial perceived lag, application developers should consider implementing a “**pre-warming**” strategy. This could involve running a trivial, dummy proof generation in a background thread upon application load, before any user interaction occurs. This would effectively pay the one-time JIT compilation cost upfront, ensuring that when the user initiates a real proof, the function is already executing in its fully optimized state.

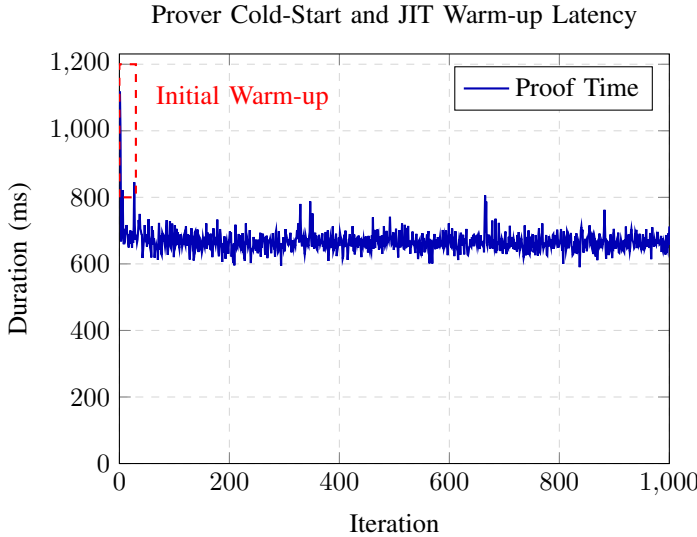


Fig. 6: The performance impact of JIT compilation warm-up observed over 1,000 consecutive runs. The first few iterations exhibit significantly higher latency before stabilizing as the execution environment is optimized.

The effect is illustrated in Figure 6, which plots the latency of 1,000 consecutive proof generation runs on a mainstream mobile device (Snapdragon 732G). The first 1-3 executions

VII. DISCUSSION AND FUTURE WORK

Our work demonstrates that the PTX protocol is a viable and efficient alternative to traditional DCV mechanisms. However, like any system, its security and applicability are contingent upon its surrounding infrastructure and open avenues for future research. This section discusses the most significant of these considerations.

A. The Role of DNSSEC in Anchor Integrity

The security of the PTX protocol’s anchor mechanism is fundamentally predicated on the integrity of the Domain Name System itself. The `Verify` algorithm’s Anchor Check (Section IV.B) implicitly trusts that the response received from a DNS query is authentic and has not been tampered with in transit. In a standard DNS environment, this assumption is not always guaranteed. An adversary with the ability to perform a man-in-the-middle attack between the verifier and the DNS resolver, or one who can compromise an upstream DNS cache, could potentially spoof the DNS response. Such an attacker could either cause a valid proof to fail verification (a denial-of-service attack) or, in a more complex attack, attempt to substitute a different metadata hash to undermine the proof’s context.

The established and standardized solution to this threat is the **Domain Name System Security Extensions (DNSSEC)** [23]. DNSSEC provides cryptographic authentication of DNS data by having zone owners sign their records with public key cryptography. A verifier’s DNS resolver can then validate this signature, creating a chain of trust that extends all the way back to the root DNS zone.

When a domain has DNSSEC enabled, a verifier’s query for the PTX anchor TXT record is no longer just a data retrieval operation; it is a verifiable cryptographic exchange. The resolver can confirm that the response originated from the authoritative nameserver for the domain and that it was not altered in transit.

Therefore, while the PTX protocol can function on any domain, its deployment as a high-security credentialing system effectively inherits the security posture of the underlying domain. For use cases requiring a level of trust equivalent to that of the TLS/PKI ecosystem, the enabling of DNSSEC on the anchor domain should be considered a mandatory operational requirement. The PTX protocol, in effect, leverages DNSSEC to transform the DNS from a simple key-value store into a cryptographically secure, decentralized public bulletin board for trust anchors.

B. Extensibility: Beyond DNS to Alternative Trust Anchors

While this paper has focused on the Domain Name System as the primary anchoring mechanism, the “Extensible” in the PTX protocol is a core architectural principle. The `trust_method` enum within the Protobuf schema is designed to allow the protocol to be adapted to other public, auditable data stores, provided they satisfy a critical requirement: **the prover must be able to write arbitrary data to a location they can deterministically derive from their secret proof inputs.** This “prover-controlled anchor” model is essential for the “unfindable anchor” privacy mechanism. This constraint opens up several compelling avenues for future work.

C. Public Key-Addressable Storage

A powerful extension lies in using key-centric decentralized systems. Many modern identity and storage protocols (e.g., Ceramic Network [24], certain implementations of DIDs [25]) allow a user, identified by a public key, to write data to a specific, addressable location in a public data store (like a key-value database or a document store).

1) Workflow: The secret in the PTX proof could be the private key of a user’s wallet. The anchor would not be a domain, but the user’s public key or Decentralized Identifier (DID). The verifier would query the public data store at the location associated with that public key to find the metadata hash. This would transform PTX from a proof of domain control to a proof of key control, making it a powerful tool for the Web3 ecosystem.

D. Ephemeral Public Notaries

While a Git commit hash is not controllable, a user can control the content of a file they own. An alternative `trust_method` could be `GIST`.

1) Workflow: The `anchor` details in the PTX file would specify a URL to a public GitHub Gist owned by the prover. The verifier would fetch the content of this Gist. The security model would require the Gist’s content to be the expected metadata hash. This is less private than the “unfindable anchor” (since the Gist URL is public), but it provides a free, universally accessible, and prover-controlled platform for anchoring attestations without needing a custom domain. The prover can still achieve revocation by simply deleting the Gist.

E. Writeable Smart Contract Slots

For blockchain applications, a user could anchor a proof to a smart contract they control.

1) Workflow: Instead of anchoring to the contract’s state, the user would anchor it to a specific, writeable mapping within the contract (e.g., `mapping(bytes32 → bytes32) public ptxAnchors`). The commitment would be the key in this mapping, and the `metadataHash` would be the value. The prover would pay a gas fee to write this anchor. A verifier would then make a gas-free `eth_call` to the contract’s public mapping to read the value in systems like Ethereum [26]. This provides an on-chain, censorship-resistant anchor that is still controlled by the prover.

These examples illustrate that the core PTX pattern is adaptable to any public system where a user has the authority to write a specific value to a specific, computable location. Future research can explore the unique security, privacy, and cost trade-offs of these alternative anchoring mechanisms.

VIII. LIMITATIONS AND POTENTIAL MITIGATIONS

While the PTX protocol offers significant advantages in flexibility, privacy, and revocation speed, its design is not without trade-offs and limitations. This section discusses the most significant of these, along with potential avenues for future work to mitigate them.

A. The Trusted Setup Requirement of Groth16

Our reference implementation utilizes the Groth16 zk-SNARK proving system due to its unparalleled proof succinctness and verifier efficiency. However, a well-known drawback of Groth16 is its requirement for a per-circuit, multi-party trusted setup ceremony. The security of the entire system is contingent upon the assumption that at least one participant in this ceremony securely destroyed their secret contribution (the “toxic waste”). While our work leverages the output of a large-scale public ceremony (the Perpetual Powers of Tau) to distribute this trust, the requirement itself remains a fundamental limitation.

1) Future Work: A promising direction for future research is to adapt the PTX protocol to use alternative ZKP schemes that feature a more desirable setup process. PLONK [27], for example, requires a universal and updateable trusted setup, meaning a single ceremony can support many different circuits. Even more compelling are transparent, “trustless” setup schemes like STARKs [20], which rely on public randomness and eliminate the need for a trusted setup entirely. However,

these schemes come with their own trade-offs, most notably a significant increase in proof size, which would impact the “portability” aspect of the PTX artifact, and the increased size of the proving key that needs to be shipped to provers, restricting accessibility. A formal analysis of these trade-offs in the context of the PTX protocol is a rich area for future work.

B. Prover Computational Cost

As our performance evaluation in Section VI demonstrates, the generation of a zk-SNARK proof is a computationally intensive operation. While achievable on modern consumer hardware, the latency can be significant, particularly on lower-end mobile devices and during the “JIT warm-up” phase in web environments. This computational burden could be a barrier to adoption for applications requiring instant, real-time proof generation from a wide range of client devices. Several architectural patterns could mitigate this limitation. One approach is to anchor the proof to a specific mapping within a contract (e.g., `ptxAnchors`) instead of the full contract state. Another is a “Prover-as-a-Service” (PaaS) model, where a client offloads proof generation to a dedicated server. This introduces trust considerations: if the server also knows the secrets, it can impersonate the user. This could be addressed using two-party computation (2PC) or recursive ZKPs, where the server assists without learning the private witness.

IX. TOWARDS A FORMAL STANDARD

The primary goal of this research has been to design and validate a novel protocol for non-interactive domain control validation. However, for any protocol to achieve widespread adoption and interoperability, it must move beyond a reference implementation to a formal, community-vetted standard. The PTX protocol, with its clearly defined file format, metadata schema, and verification logic, is an ideal candidate for such standardization.

The history of internet infrastructure has shown that the most successful protocols are those built on open standards, such as the Request for Comments (RFCs) published by the Internet Engineering Task Force (IETF). These documents provide an unambiguous blueprint that allows for the development of multiple, independent, yet fully interoperable implementations. A formal RFC for the PTX protocol would need to precisely specify:

- 1) The binary structure of the `.ptx` file, including the magic header and the canonical encoding of the Protobuf messages.
- 2) The requirements of the `SignedMetadataPayload`, including mandatory validation of the `purpose` fields.
- 3) The exact cryptographic suite, including the chosen elliptic curve (e.g., BN254 [13]), the zk-SNARK scheme (e.g., Groth16 [2]), and the ZK-friendly hash function (e.g., Poseidon [3] with specified parameters).
- 4) The deterministic algorithms for deriving the anchor hostname and the expected TXT record value.

A. Future Work

A significant next step for this project is to draft a formal specification for the PTX protocol and submit it as an Internet-Draft to the IETF. This would begin the process of community review, refinement, and potential adoption as a standard. Achieving RFC status would provide a stable foundation for browser vendors, certificate authorities, and developers of decentralized applications to build interoperable tools, transforming PTX from a research artifact into a fundamental building block of the web’s trust infrastructure.

APPENDIX ARTIFACT AVAILABILITY

The reference implementation of the PTX protocol, including the Go toolchain used for benchmarking and the Circom circuit artifacts, is publicly available under the Apache 2.0 License. The repository contains instructions for reproducing the performance benchmarks presented in Section VI.

The artifact can be found at: <https://github.com/Stygian-Inc/ptx-jesuit-go>

Additionally, the circuits used in this paper are available at <https://github.com/Stygian-Inc/PTX-Circuit-PoC>

ACKNOWLEDGMENT

The author would like to thank the developers of the `gnark` and `snarkjs` libraries, whose work on zero-knowledge proof systems made this research possible.

REFERENCES

- [1] R. Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten, “Automatic Certificate Management Environment (ACME),” RFC 8555, Mar. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8555>
- [2] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305–326.
- [3] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A new hash function for Zero-Knowledge proof systems,” in *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*. <https://www.usenix.org/system/files/sec21-grassi.pdf>, 2021, pp. 519–535. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
- [4] E. Rescorla, “The Transport Layer Security (TLS) protocol version 1.3,” Internet Requests for Comments, RFC Editor, RFC 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8446>
- [5] H. Booth, M. Ogata, K. Kent, M. Souppaya, and D. Dodson, “Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities,” National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST SP 800-218, Feb. 2022. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [6] J. Aas, R. Barnes, B. Case, Z. Durumeric, P. Eckersley, A. Flores-López, J. A. Halderman, J. Hoffman-Andrews, J. Kasten, E. Rescorla, S. Schoen, and B. Warren, “Let’s encrypt: An automated certificate authority to encrypt the entire web,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS ’19)*. New York, NY, USA: Association for Computing Machinery, 2019, pp. 2473–2487.
- [7] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” Internet Requests for Comments, RFC Editor, RFC 5280, May 2008. [Online]. Available: <https://www.rfc-editor.org/info/rfc5280>
- [8] S. Bowe, A. Gabizon, and I. Miers, “Scalable multi-party computation for zk-snark parameters,” in *International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. Springer, 2017, pp. 45–75.

- [9] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, "CIRCOM: A circuit description language for building zero-knowledge applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4743, 2022.
- [10] P. V. Mockapetris, "Domain names - concepts and facilities," Internet Requests for Comments, RFC Editor, RFC 1034, Nov. 1987. [Online]. Available: <https://www.rfc-editor.org/info/rfc1034>
- [11] Google, "Protocol buffers," 2008. [Online]. Available: <https://protobuf.dev/>
- [12] M. B. Jones, J. Bradley, and N. Sakimura, "JSON web signature (JWS)," RFC 7515, RFC Editor, RFC 7515, May 2015. [Online]. Available: <https://www.rfc-editor.org/info/rfc7515>
- [13] P. S. L. M. Barreto and M. Naehrig, "Pairing-friendly elliptic curves of prime order," *Selected Areas in Cryptography*, pp. 319–331, 2005.
- [14] K. Harrenstien, M. Stahl, and E. J. Feinler, "Dod internet host table specification," Internet Requests for Comments, RFC Editor, RFC 952, Oct. 1985. [Online]. Available: <https://www.rfc-editor.org/info/rfc952>
- [15] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols," in *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93)*. ACM, 1993, pp. 62–73.
- [16] G. Botrel, T. Piellard, Y. E. Housni, A. Tabaie, and I. Kubjas, "Consensys/gnark: High-performance zk-SNARK library in Go," 2025, latest major updates as of December 2025. [Online]. Available: <https://github.com/Consensys/gnark>
- [17] iden3, "websnark: WebAssembly-based zk-SNARK prover and verifier," GitHub Repository, 2019, part of the iden3 project, licensed under GPL-3.0. Accessed December 2025. [Online]. Available: <https://github.com/iden3/websnark>
- [18] S. Bowe, J. Grigg, and D. Hopwood, "Recursive proof composition without a trusted setup," Cryptology ePrint Archive, Paper 2019/1021, 2019, <https://eprint.iacr.org/2019/1021>. [Online]. Available: <https://eprint.iacr.org/2019/1021>
- [19] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 315–334.
- [20] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "STARK: A scalable and transparent argument of knowledge," in *Advances in Cryptology – CRYPTO 2018*, ser. Lecture Notes in Computer Science, vol. 10991. Springer, 2018, pp. 33–63.
- [21] iden3, "snarkjs: zk-SNARK implementation in JavaScript and WebAssembly," GitHub Repository, 2021, accessed: December 2025. Supports Groth16, PLONK, and FFLONK. [Online]. Available: <https://github.com/iden3/snarkjs>
- [22] Google, "V8 JavaScript engine," 2008, latest major release as of late 2025. [Online]. Available: <https://v8.dev/>
- [23] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS security introduction and requirements," RFC 4033, 2005.
- [24] 3Box Labs, "Ceramic protocol specification," GitHub Repository, 2020, accessed: December 2025. [Online]. Available: github.com
- [25] M. Sporny, A. Guy, M. Sabadello, and D. Reed, "Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations," World Wide Web Consortium (W3C), W3C Recommendation, Jul. 2022, available at <https://www.w3.org/TR/did-1.0/>. [Online]. Available: <https://www.w3.org/TR/2022/REC-did-core-20220719/>
- [26] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Yellow Paper, 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [27] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Report 2019/953, 2019, <https://eprint.iacr.org/2019/953>.